

单位代码： 10293 密 级： 公开

南京邮电大学

专业学位硕士论文



论文题目： 面向长尾恶意软件家族分类的
条件生成增强 MalConv 框架研究

学 号	1025040926
姓 名	沈文强
导 师	何静
专业学位类别	网络空间安全
学 院 名 称	网络空间安全学院
类 型	全日制专业学位硕士
专 业 （领 域）	网络空间安全
论文提交日期	2026 年 6 月

摘要

恶意软件家族分类是恶意样本分析、威胁归因与自动化响应中的关键基础任务。基于字节级的端到端建模方法（如 MalConv）能够直接处理原始可执行文件，避免人工特征工程引入的信息损失，已成为该任务中具有代表性的技术路线。然而，真实安全数据通常呈现显著的长尾分布：以 Microsoft Malware Classification Challenge 数据集为例，最大类 Kelihos_ver3 与最小类 Simda 的样本数比例约为 70 : 1，少数类在测试集中可能仅有 3 至 6 个样本，致使模型即便取得 93% 以上的总体准确率，仍难以充分识别极少数类家族。

针对上述问题，本文围绕“条件生成增强能否稳定提升长尾恶意软件家族分类的少数类性能”这一研究问题，提出一种面向字节级长尾场景的条件 GAN 增强 MalConv 框架。该框架以多尺度门控卷积 MalConv 为骨干分类器，引入类别条件生成器，在嵌入空间通过梯度惩罚与特征匹配机制进行对抗训练，并设计基于分类置信度的合成样本过滤策略，仅保留高置信度生成样本用于后续增强微调。为保证实验结论可信，本文同步构建了包括多随机种子训练、逐类指标导出、消融分析与检查点审计在内的可复现实验协议，并对历史训练产物进行了系统审计，确认旧版 GAN 实现存在“分类张量未更新即被评估”这一致命缺陷，从根源上修复了实验设计错位。

在 10 个随机种子 (42, 52, 62, 72, 82, 92, 102, 112, 122, 132) 下的扩展实验表明：所提 GAN + Finetune 管线相较其直接长尾基线 focal_sampler，宏平均 F1 由 0.8656 ± 0.0261 提升至 0.8678 ± 0.0173 ，差值 +0.0022（配对 t 检验 $p=0.731$ ，Cohen's $d = +0.12$ ），未达统计显著性，但跨种子标准差下降约 34%，呈现一致的稳定性改进；其均值性能未超过更简单的 raw_malconv 基线 0.8820 ± 0.0307 （差值 -0.0142，Wilcoxon $p=0.010$ ），后者反而在 7 个类别上取得 10 种子均值最优。消融分析进一步显示，冻结骨干网络是当前管线最关键的稳定性因素（去除冻结后宏 F1 下降约 0.05），而特征匹配、梯度惩罚与置信度过滤对总体均值的影响有限。最终，本文将所提框架定位为“长尾恶意软件分类中的生成增强机制研究工具”：其价值不在于刷新单一指标最优，而在于通过多种子、多指标、统计显著性与稳定性多维评估，系统刻画 GAN 在安全长尾场景下的真实行为边界，并为面向部署稳定性的对抗增强方法设计提供可复现的实证基础。

关键词：恶意软件家族分类，长尾学习，条件生成对抗网络，MalConv，可复现性研究

Abstract

Malware family classification is a foundational task for malicious sample analysis, threat attribution and automated incident response. Byte-level end-to-end models such as MalConv directly consume raw executable contents and avoid information loss caused by manual feature engineering, making them a representative technical route in this field. However, real-world security data are typically long-tailed: in the Microsoft Malware Classification Challenge dataset the largest family Kelihos_ver3 contains roughly 70 times more samples than the smallest family Simda, and the latter may have only 3 to 6 instances in the held-out test set, so even a model with above 93% overall accuracy can still fail to recognise rare families.

Motivated by this gap, this thesis investigates whether conditional generative augmentation can reliably improve minority-class performance in long-tailed malware family classification. A conditional-GAN-enhanced MalConv framework tailored to byte-level long-tailed settings is proposed. A multi-scale gated convolutional MalConv serves as the backbone classifier; a class-conditional generator is trained adversarially in the embedding space with gradient penalty and feature matching; generated samples are further filtered by classifier confidence and used to fine-tune the classifier in an augmented training stage. To support credible empirical claims, a reproducible experimental protocol is constructed, including multi-seed training, per-class metric export, ablation studies and checkpoint auditing. A systematic audit of legacy artefacts reveals a critical flaw in an earlier GAN implementation, in which the classification tensors loaded by the evaluation script were never actually updated during adversarial training, and this design error is fixed at its root.

Across ten random seeds (42, 52, 62, 72, 82, 92, 102, 112, 122, 132), the proposed GAN + Fine-tune pipeline raises macro-F1 from 0.8656 ± 0.0261 (focal_sampler baseline) to 0.8678 ± 0.0173 , a difference of $+0.0022$ (paired-t $p=0.731$, Cohen’s $d = +0.12$) that is not statistically significant, while the across-seed standard deviation drops by roughly 34%, indicating a consistent stability gain. The pipeline does not surpass the simpler raw_malconv baseline (0.8820 ± 0.0307 ; -0.0142 , Wilcoxon $p=0.010$), which in fact achieves the best ten-seed mean on seven of the nine families. Ablation studies show that freezing the backbone during adversarial training is the most decisive stability factor (removing it drops macro-F1 by about 0.05), whereas feature matching, gradient penalty and confidence filtering have only marginal effects on the mean. The proposed framework is therefore positioned as a reproducible research tool for studying generative augmentation in long-tailed security data: its main contribution is not chasing a single best score, but characterising the realistic behavioural boundary of GAN-based augmentation through multi-seed, multi-metric, significance

and stability analyses, and providing an evidence-based foundation for deployment-stable adversarial augmentation methods.

Key Words: Malware Family Classification, Long-Tailed Learning, Conditional GAN, MalConv, Reproducibility Study

目 录

摘要	I
Abstract	II
插图索引	VII
表格索引	VIII
专用术语注释表	IX
第一章 绪论	1
1.1 研究背景及意义	1
1.2 国内外研究现状	2
1.2.1 字节级恶意软件检测与分类	2
1.2.2 长尾学习与类别再加权	2
1.2.3 基于 GAN 的数据增强	2
1.2.4 安全数据上的生成对抗与对抗样本研究	3
1.2.5 可复现性与统计严谨性	3
1.3 本文的主要工作及创新点	3
1.4 论文章节安排	4
第二章 相关背景知识介绍	5
2.1 字节级恶意软件分类模型 MalConv	5
2.1.1 字节嵌入与门控卷积	5
2.1.2 多尺度池化与分类	5
2.2 条件生成对抗网络与训练稳定化	6
2.2.1 原始 GAN 与条件 GAN	6
2.2.2 梯度惩罚与特征匹配	6
2.2.3 Gumbel-Softmax 与离散符号生成	7
2.3 长尾学习与不均衡分类	7
2.3.1 类别再加权与困难样本损失	7
2.3.2 生成式数据增强	8
2.4 统计显著性与可复现性分析方法	8
2.4.1 多随机种子评估	8
2.4.2 配对显著性检验	8
2.4.3 效应量分析	8
2.5 本章小结	9
第三章 条件 GAN 增强 MalConv 框架设计与实现	10
3.1 框架总览	10
3.1.1 设计原则	10
3.1.2 三阶段流程	10
3.2 字节级 MalConv 骨干	11
3.2.1 超长字节序列的处理	11
3.2.2 多尺度门控卷积的判别意义	11
3.3 类条件字节生成器	11
3.3.1 生成器结构	11
3.3.2 为何不直接生成 50000 字节	12

3.4	辅助分类判别器与对抗训练损失	12
3.4.1	共享骨干的双头判别器	12
3.4.2	对抗训练损失	12
3.4.3	冻结骨干的判别器更新策略	13
3.5	基于置信度过滤的增强微调	13
3.5.1	合成样本筛选	13
3.5.2	按类目标数补齐	14
3.5.3	微调阶段参数设置	14
3.6	完整算法描述	14
3.7	本章小结	15
第四章	实验设置与可复现实验协议	16
4.1	数据集与类别分布	16
4.1.1	Kaggle Microsoft Malware Classification Challenge 数据集	16
4.1.2	长尾分布特性	16
4.1.3	数据划分与预处理	17
4.2	对比方法	17
4.2.1	核心对比方法	17
4.2.2	消融变体	17
4.3	评价指标	18
4.3.1	总体与宏平均指标	18
4.3.2	极少数类专项指标	18
4.4	超参数与训练硬件	18
4.4.1	主要超参数	18
4.4.2	训练硬件与软件环境	18
4.5	多种子可复现实验协议	19
4.5.1	种子规模与统一入口	19
4.5.2	结果聚合与产物归档	20
4.5.3	历史实验工件审计	20
4.5.4	统计检验协议	20
4.6	写作约束	20
4.7	本章小结	21
第五章	实验结果与分析	22
5.1	历史实验工件审计	22
5.1.1	张量级哈希比对	22
5.1.2	问题根因	22
5.1.3	修复措施	22
5.2	10 种子核心结果	23
5.2.1	总体指标	23
5.2.2	结果分析	24
5.3	逐类 F1 详细分析	24
5.3.1	多数类表现稳定	26
5.3.2	中等类的可重复正向增益	26
5.3.3	极少数类 Simda 的负向漂移	26
5.3.4	生成增强相较 raw_malconv 的相对劣势	26

5.4	极少数类 Simda 专项分析	26
5.5	统计显著性与效应量分析	28
5.6	跨种子方差与“均值-方差”权衡	29
5.7	训练动态与混淆矩阵差异分析	30
5.7.1	训练动态	30
5.7.2	混淆矩阵差异	31
5.8	消融实验结果	33
5.9	讨论：生成增强的收益边界	34
5.10	本章小结	34
第六章	总结与展望	36
6.1	本文工作总结	36
6.2	进一步研究方向	37
参考文献		39
附录 A	程序清单	41
附录 B	攻读专业学位硕士期间撰写的论文	43
附录 C	攻读专业学位硕士期间申请的专利	44
附录 D	攻读专业学位硕士期间参加的科研项目	45
致谢		46

插图索引

图 5.1 Microsoft MMCC 任务中九个恶意软件家族在训练集的样本分布(数据源:analyze_experiments.py 输出)	23
图 5.2 核心方法多指标雷达图对比(数据源:analyze_experiments.py 输出 results/analysis/radar_chart.png)	25
图 5.3 核心方法在各恶意软件家族上的逐类平均 F1 热力图 ($n=10$ 种子均值; 数据源: results/analysis/heatmap_core.png)	25
图 5.4 各方法在 Simda 家族的 F1 对比(数据源: results/analysis/simda_f1_comparison.png)	27
图 5.5 各方法 macro-F1 在 10 个随机种子上的分布对比(数据源:results/analysis/seed_variance_boxplot.png)	29
图 5.6 GAN 对抗训练阶段判别器损失、生成器损失、特征匹配损失及验证 macro-F1 随 epoch 变化(数据源: results/analysis/training_dynamics.png)	30
图 5.7 增强微调阶段的训练与验证曲线(数据源: results/analysis/finetune_dynamics.png)	31
图 5.8 代表性基线方法的训练收敛曲线示例(数据源: results/analysis/baseline_training_curves.png)	31
图 5.9 gan_pipeline (Finetune) 与 focal_sampler 混淆矩阵差异热力图(3 种子均值; 数据源: results/analysis/confusion_diff_ft_vs_focal.png)	32
图 5.10 消融变体与各阶段模型在各恶意软件家族上的逐类平均 F1 热力图 ($n=3$ 种子均值; 数据源: results/analysis/heatmap_ablations.png)	33

表格索引

表 3.1 MalConv 骨干网络主要超参数 11

表 4.1 Microsoft MMCC 数据集 9 类家族样本分布 16

表 4.2 消融实验变体设计 18

表 4.3 核心实验超参数取值 19

表 5.1 历史 checkpoint 张量级比对结果 22

表 5.2 核心方法 10 种子总体指标 (mean ± std, n = 10) 23

表 5.3 核心方法逐类 F1 均值 (n = 10), 加粗为该列方法间最高值 24

表 5.4 各方法在 Simda 上的 F1 统计 (n = 10) 27

表 5.5 Simda 类逐种子 F1 (节选) 27

表 5.6 核心方法对的统计显著性与效应量 (n = 10) 28

表 5.7 各方法 macro-F1 与 macro-Recall 跨种子标准差 (n = 10) 29

表 5.8 消融实验结果 (n = 3, 均值 ± 标准差) 33

专用术语注释表

符号说明：

符号	表示含义
x	原始字节
y	恶意软件
C	家族类别
L_{max}	MalConv
L_{gen}	生成器输
z	生成器输
d_z	噪声维度
$G(z, y)$	类条件生
$D(\cdot)$	判别器 (
$f(\cdot)$	最终用于
λ_{GP}	梯度惩罚
λ_{FM}	特征匹配
τ	Gumbel-S
τ_{conf}	增强样本
N_{aug}	增强阶段
σ	跨随机种
d	Cohen's κ

缩略词说明：

缩写	英文全称
AC-GAN	Auxiliary
APT	Advanced
AUC	Area Und
BN	Batch No
CE	Cross Ent
CNN	Convolut

cGAN	Conditional Generative Adversarial Network
FM	Feature Map
GAN	Generative Adversarial Network
GP	Gradient Descent
IAT	Importance Sampling
IDS	Intrusion Detection System
JSON	JavaScript Object Notation
LSTM	Long Short-Term Memory
MalConv	Malware Convolutional Neural Network
ML	Machine Learning
PE	Portable Executable
PR	Precision-Recall Curve
ReLU	Rectified Linear Unit
ROC	Receiver Operating Characteristic
SGD	Stochastic Gradient Descent
SOTA	State of the Art
WGAN-GP	Wasserstein Generative Adversarial Network with Gradient Penalty

第一章 绪论

1.1 研究背景及意义

近年来，恶意软件数量与变种规模持续增长。据 AV-TEST 公布的统计，恶意样本总量已达数十亿量级，每日新增样本数以十万计^[1]。此类样本在企业网络、关键信息基础设施与个人终端上构成持续性威胁，并成为高级持续性威胁（Advanced Persistent Threat, APT）、勒索软件攻击与供应链攻击等复杂攻防场景中的基础载体。在这种规模化对抗背景下，恶意软件检测与家族归类已成为网络空间安全态势感知与自动化响应体系中最核心的支撑能力之一^[2]。

恶意软件家族分类（malware family classification）的目标是在确认样本为恶意的基础上，进一步将其归入对应家族，以便分析人员快速定位变种来源、复用既有逆向分析结论并选择针对性的处置策略。相较二分类的“良性/恶意”判别，家族分类需要刻画更精细的类内一致性与类间差异性，对模型的判别能力要求更高，因而成为恶意代码静态分析与威胁情报分析领域的重要研究主题。

在分类技术路线上，传统方法主要依赖 API 调用序列、字节 n -gram、控制流图或人工设计的统计特征^[2]，需要较多的工程经验与逆向知识，且容易因加壳、混淆等对抗手段失效。相比之下，基于原始字节序列的端到端深度模型不依赖任何手工特征，能够直接处理可执行文件（PE 文件）的原始内容。其中，Raff 等人提出的 MalConv^[3] 是该路线的代表性工作，其以门控卷积与全局池化结构直接处理超长字节序列，并在多项公开评测中展示出较强的分类能力，已被工业界与学术界广泛复现与扩展^[4]。

然而，真实安全数据通常呈现显著的长尾分布特征：少数活跃家族占据样本总量的绝大部分，而大量低活跃或新兴家族的样本数量稀缺。以 Microsoft Malware Classification Challenge（下称“Kaggle MMCC”数据集）为例^[5]，本文实际统计显示：最大类 Kelihos_ver3 含 2942 个样本，最小类 Simda 仅含 42 个样本，最大类与最小类样本比约为 70 : 1，且后者在 15% 比例的测试集中通常仅有 3 至 6 个样本。在这种极端不均衡设定下，分类器即便能取得 93% 以上的总体准确率，仍可能在少数类家族上出现召回率不足的情况，导致针对长尾家族的检测能力被显著低估。

面向长尾分类问题，已有研究主要从三类思路展开：基于类别再加权的损失重构（如加权交叉熵、Focal Loss^[6]）、基于采样的均衡化方法（如 SMOTE 类过采样、欠采样）以及基于数据生成的样本增强方法^[7]。其中，基于条件生成对抗网络（Conditional GAN, cGAN）^[8,9] 与辅助分类器生成对抗网络（AC-GAN）^[10] 的生成增强方法，因能针对指定类别合成样本而被广泛认为是长尾分类的潜在突破口。然而，将 GAN 类方法直接迁移到字节级恶意软件分类场景仍面临三方面困难：第一，恶意软件样本表现为超长、离散的字节序列，难以直接套

用图像/连续向量上的对抗训练框架；第二，类条件生成器在少样本类别（如本文 42 个样本的 Simda 家族）上几乎无法获得稳定的训练信号；第三，缺乏面向多种子、多种指标的可重复评估协议，致使既有报告往往因单次实验波动而出现“看似显著、实则非鲁棒”的结论。

综合上述背景，研究“在严谨的可复现实验协议下，条件生成增强能否切实改善长尾恶意软件家族分类（尤其是少数类）的性能”这一问题，既有方法学层面的科学价值，也有面向安全部署的工程价值。其方法学价值在于揭示生成增强在字节级离散安全数据中的真实有效性边界，避免在工业部署中盲目复现“图像 GAN 增强”的范式；其工程价值在于为面向部署稳定性的检测系统设计提供可量化的“均值-方差”权衡依据，从而支撑更可靠的家族级威胁归因。

1.2 国内外研究现状

1.2.1 字节级恶意软件检测与分类

字节级恶意软件建模的代表性工作 MalConv^[3] 通过嵌入层、门控卷积与全局最大池化结构直接处理可执行文件的原始字节序列，避免了传统方法中复杂的特征工程。在此之上，Krčál 等人提出的 DeepConv 系列模型^[11] 在大规模工业数据集上验证了字节卷积的可扩展性；Coull 与 Gardner^[12] 进一步分析了 MalConv 内部激活与可解释性；Anderson 与 Roth^[4] 公开的 EMBER 数据集与字节特征基线则推动了该方向的可复现研究。最新工作如 MalConv2^[13] 通过时序池化与局部注意力机制对 MalConv 进行了改进，但整体路线仍延续“原始字节 + 长程卷积”的设计思想。

1.2.2 长尾学习与类别再加权

长尾视觉识别催生了大量类别均衡技术。Lin 等人提出的 Focal Loss^[6] 通过动态因子放大困难样本梯度，已成为不平衡分类的标准基线之一；Cui 等人提出的“有效样本数”加权^[14] 显式建模了类别样本量与损失权重之间的非线性关系；Cao 等人提出的 LDAM 损失^[15] 进一步在分类边界上引入类别相关的间隔约束。Zhang 等人^[7] 对长尾学习进行了系统综述，指出在极少数类（few-shot tail）场景下，单纯的重加权或重采样往往收益有限，必须结合表示学习或数据扩充才能获得稳健改进。

1.2.3 基于 GAN 的数据增强

条件生成对抗网络^[8,9] 通过引入类别条件，使生成器能够按需合成指定类别的样本。Odena 等人提出的 AC-GAN^[10] 进一步将分类损失耦合到判别器，使生成与分类任务在同一网络中

协同优化；Salimans 等人提出的特征匹配 (Feature Matching) ^[16] 通过约束生成样本与真实样本在判别器中间层的特征统计一致，缓解了 GAN 训练的不稳定性；Gulrajani 等人在 WGAN-GP^[17] 中引入梯度惩罚替代权重裁剪，使训练稳定性得到进一步保障；Jang 等人^[18] 提出的 Gumbel-Softmax 重参数化为离散符号的可微生成提供了通用工具。

1.2.4 安全数据上的生成对抗与对抗样本研究

在安全场景下，Hu 与 Tan 提出的 MalGAN^[19] 通过 GAN 生成能够绕过黑盒检测器的恶意样本特征向量，是 GAN 在恶意软件领域的标志性工作之一。其后续工作如 GAPGAN^[20]、Adversarial Malware Examples^[21] 等则进一步探索了基于强化学习或对抗扰动的可执行文件级逃逸样本生成。然而，这些工作的目标多为“生成对抗样本以攻击检测器”，而非“生成增强样本以训练分类器”；将类条件生成应用于多类家族分类增强的研究仍较为有限，且大多缺乏多种子统计与系统化的消融分析。

1.2.5 可复现性与统计严谨性

随着机器学习领域对可复现危机^[22,23] 的关注，越来越多工作开始强调多随机种子评估、配对统计检验与效应量分析。在安全机器学习社区，Arp 等人^[24] 总结了恶意软件检测研究中常见的方法学陷阱，并明确建议在评估时报告跨种子方差与统计显著性。本文的实验协议正是基于这一方向构建，并将其作为论文的核心方法学贡献之一。

1.3 本文的主要工作及创新点

针对长尾恶意软件家族分类中的“类别极端不均衡”与“生成增强收益不可重复”两个核心问题，本文主要工作及创新点如下：

- (1) 提出面向长尾恶意软件家族分类的条件 GAN 增强 MalConv 统一框架。该框架以多尺度门控卷积 MalConv 为骨干，引入类别条件软字节生成器与共享骨干的辅助分类判别器，并在嵌入空间施加梯度惩罚与特征匹配约束，实现了对抗训练与分类任务在同一网络中的稳定协同优化。
- (2) 设计基于分类置信度的合成样本过滤机制与按类目标数补齐策略。在 GAN 训练完成后，利用当前 MalConv 对生成样本进行预测，仅保留类别预测正确且置信度高于阈值 τ_{conf} 的样本，并按 $N_{aug} = 800$ 的每类目标数对训练集进行均衡补齐，最后在增强训练集上以小学习率执行微调，使生成样本服务于少数类补强而非无差别扩增。

- (3) 构建包含多随机种子训练、逐类指标导出、关键组件消融分析与检查点审计在内的可复现实验协议。本文通过 `audit_checkpoints.py` 工具对历史模型权重进行张量级比对, 定位并修复了旧版 GAN 实现中“分类张量未真正更新即被评估”这一致命缺陷; 随后在 10 个随机种子 (42, 52, 62, 72, 82, 92, 102, 112, 122, 132) 下重新评估了 5 类核心方法与 5 类消融变体, 并辅以配对 t 检验、Wilcoxon 符号秩检验与 Cohen's d 效应量分析。
- (4) 从“均值-方差”双视角刻画生成增强的真实收益边界。本文在 10 种子下实证发现: GAN + Finetune 管线相较直接基线 `focal_sampler` 的宏 F1 均值改善 (+0.0022) 未达统计显著性, 且其均值仍低于更简单的 `raw_malconv` 基线 (-0.0142 , Wilcoxon $p=0.010$); 但其跨种子标准差下降约 34%–59%, 呈现一致的稳定性优势。该结论修正了既有研究在 3 种子小样本下报告“GAN 增强显著提升少数类”的乐观偏差, 为面向部署稳定性的安全场景方法选型提供了更可靠的实证基础。

1.4 论文章节安排

本文共分六章, 章节安排如下:

第一章为绪论, 介绍研究背景与意义、国内外研究现状以及本文的主要工作与创新点。

第二章为相关背景知识介绍, 分别阐述 MalConv 字节级建模、条件生成对抗网络与训练稳定化技巧、长尾学习与不均衡分类方法, 以及本文涉及的统计显著性分析方法。

第三章详细介绍本文提出的条件 GAN 增强 MalConv 框架, 包括字节级 MalConv 骨干、类条件字节生成器、辅助分类判别器、训练损失设计、基于置信度的合成样本过滤与增强微调流程, 并给出整体算法描述。

第四章给出实验设置与可复现实验协议, 包括数据集划分、对比方法、评价指标、超参数配置、多种子方案、消融实验设计与统计检验方法。

第五章给出系统的实验结果与分析, 包括历史实验工件审计、10 种子核心结果、逐类 F1 与极少数类 Simda 专项分析、统计显著性与效应量分析、训练动态分析、跨种子方差分析、混淆矩阵差异分析以及消融实验结果。

第六章总结全文工作并展望后续研究方向。

第二章 相关背景知识介绍

本章对本文工作所依赖的关键技术与理论基础进行系统梳理，包括字节级恶意软件检测中具有代表性的 MalConv 模型、条件生成对抗网络及其训练稳定化技巧、长尾学习与不均衡分类方法，以及面向多种子小样本评估的统计显著性分析方法。所介绍的内容均与第三章的方法设计和第四、五章的实验协议存在直接关联。

2.1 字节级恶意软件分类模型 MalConv

MalConv^[3] 是一种面向原始字节序列的端到端卷积神经网络，其核心思想是放弃手工特征工程，直接以可执行文件的整段字节流作为输入，通过门控卷积与全局池化提取判别模式。其结构主要包含字节嵌入层、多尺度门控卷积层、全局最大池化层以及全连接分类层。

2.1.1 字节嵌入与门控卷积

记输入字节序列为 $x \in \{0, 1, \dots, 255, 256\}^{L_{max}}$ ，其中 256 表示填充符号。MalConv 首先通过嵌入矩阵 $E \in \mathbb{R}^{257 \times d_e}$ 将其映射至 d_e 维连续空间：

$$\mathbf{X}_{emb} = \text{Embedding}(x; E) \in \mathbb{R}^{L_{max} \times d_e}. \quad (2.1)$$

随后通过卷积核大小为 k 、步长为 k 的一维卷积进行非重叠局部模式抽取。门控卷积^[25]通过两路平行卷积输出乘积形式融合，使每个通道的激活既包含模式响应也包含其门控强度：

$$\mathbf{H}^{(k)} = \text{Conv1d}_k^{feat}(\mathbf{X}_{emb}^\top) \odot \sigma(\text{Conv1d}_k^{gate}(\mathbf{X}_{emb}^\top)), \quad (2.2)$$

其中 $\sigma(\cdot)$ 为 Sigmoid 函数， $\odot$ 表示逐元素相乘。

2.1.2 多尺度池化与分类

为同时覆盖不同粒度的字节模式，本文实现采用 $k \in \{256, 512, 1024\}$ 的多尺度门控卷积，并对每一尺度的输出执行全局自适应最大池化：

$$\mathbf{h}^{(k)} = \text{AdaptiveMaxPool}(\mathbf{H}^{(k)}) \in \mathbb{R}^{C_{ch}}. \quad (2.3)$$

将不同尺度的池化向量拼接得到融合特征 $\mathbf{h} = [\mathbf{h}^{(256)}; \mathbf{h}^{(512)}; \mathbf{h}^{(1024)}] \in \mathbb{R}^{3C_{ch}}$, 再经两层带 Batch Normalization 与 ReLU 的全连接层与 Dropout 得到分类 logits:

$$\mathbf{z}_{cls} = W_2 \text{ReLU}(\text{BN}(W_1 \text{BN}(\mathbf{h}))), \quad \hat{y} = \arg \max_c \mathbf{z}_{cls}^{(c)}. \quad (2.4)$$

MalConv 的优势在于其对长序列与超长上下文的天然适配能力, 以及不依赖任何外部解析器 (如 PE 解析、API 调用追踪) 的端到端特性, 这使其特别适合作为字节级长尾家族分类的统一骨干。其局限性在于全局最大池化可能丢失局部时序信息, 且训练时需要较大的内存与显存预算。

2.2 条件生成对抗网络与训练稳定化

2.2.1 原始 GAN 与条件 GAN

Goodfellow 等人提出的生成对抗网络 (GAN) [9] 由生成器 G 与判别器 D 构成, 两者在一个零和博弈中交替更新:

$$\min_G \max_D \mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log (1 - D(G(z)))]. \quad (2.5)$$

Mirza 等人提出的条件 GAN [8] 通过将类别标签 y 拼接到生成器与判别器的输入, 使生成器能够按指定类别生成样本, 为按类增强提供了基础工具。Odena 等人提出的 AC-GAN [10] 进一步将分类损失耦合到判别器中: 判别器输出双头预测, 一个用于真假判别, 另一个用于类别预测, 从而使分类任务能够受益于对抗训练中的分布对齐压力。

2.2.2 梯度惩罚与特征匹配

GAN 训练在实践中常面临梯度爆炸、模式崩溃与训练发散等不稳定问题 [16]。本文采用两类经典稳定化技巧:

(1) **梯度惩罚 (Gradient Penalty, GP)**。Gulrajani 等人在 WGAN-GP [17] 中通过 1-Lipschitz 约束的软实现, 将判别器对插值样本的梯度范数约束在 1 附近:

$$\mathcal{L}_{GP} = \lambda_{GP} \cdot \mathbb{E}_{\hat{x}} \left[\left(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1 \right)^2 \right], \quad (2.6)$$

其中 $\hat{x} = \epsilon x + (1 - \epsilon)\tilde{x}$, $\epsilon \sim U[0, 1]$, x 与 $\tilde{x}$ 分别为真实样本与生成样本。本文将该约束施加在 MalConv 嵌入空间的插值上, 详见第三章。

(2) **特征匹配 (Feature Matching, FM)**。Salimans 等人提出的 FM 损失^[16] 通过约束生成样本与真实样本在判别器中间层激活统计上的一致性, 降低生成器盲目追逐判别器决策边界的不稳定性:

$$\mathcal{L}_{FM} = \lambda_{FM} \cdot \left\| \mathbb{E}_x[\phi(x)] - \mathbb{E}_z[\phi(G(z))] \right\|_2^2, \quad (2.7)$$

其中 $\phi(\cdot)$ 表示判别器中间层激活提取算子。

2.2.3 Gumbel-Softmax 与离散符号生成

字节序列属于离散符号空间, 直接通过 $\arg \max$ 输出会破坏梯度。Jang 等人提出的 Gumbel-Softmax 重参数化^[18] 通过引入 Gumbel 噪声与温度 τ 实现离散分布的可微近似:

$$\tilde{y}_i^{(b)} = \frac{\exp((\log \pi_i^{(b)} + g_i^{(b)})/\tau)}{\sum_j \exp((\log \pi_j^{(b)} + g_j^{(b)})/\tau)}, \quad (2.8)$$

其中 $g_i^{(b)} \sim \text{Gumbel}(0, 1)$, $\pi_i^{(b)}$ 为生成器输出的 softmax 概率。本文生成器在 Gumbel-Softmax 输出上通过线性组合得到“软嵌入”后送入判别器, 使梯度能够回传至生成器参数。

2.3 长尾学习与不均衡分类

2.3.1 类别再加权与困难样本损失

类别再加权是处理不均衡分类最直接的手段。给定类别样本数 n_c , 最朴素的反频加权交叉熵记为:

$$\mathcal{L}_{wce} = - \sum_{c=1}^C w_c \cdot y_c \log \hat{y}_c, \quad w_c \propto 1/n_c. \quad (2.9)$$

Cui 等人提出的“有效样本数”加权^[14] 进一步引入超参数 β , 使加权方案在样本数差异较大时更具数值稳定性。Lin 等人提出的 Focal Loss^[6] 通过动态因子 $(1 - \hat{y}_c)^\gamma$ 放大困难样本梯度:

$$\mathcal{L}_{focal} = - \sum_{c=1}^C w_c \cdot (1 - \hat{y}_c)^\gamma \cdot y_c \log \hat{y}_c. \quad (2.10)$$

本文在 focal_sampler 基线中采用 $\gamma = 2.0$ 的 Focal Loss, 并辅以 PyTorch 的 WeightedRandomSampler 对小批量构造进行重采样。

2.3.2 生成式数据增强

当类别样本数过少时,单纯的再加权可能导致过拟合极少数训练样本。生成式数据增强通过为少数类合成新样本来扩充训练分布。在图像领域,条件 BigGAN 等模型已可在 ImageNet-LT 等长尾基准上实现可观提升^[7]。然而在恶意软件字节序列场景下,生成式增强需要同时满足三个条件: (i) 按类条件生成, (ii) 输出可被分类器消费的离散字节, (iii) 不破坏分类器已有判别能力。本文在第三章中通过类条件软字节生成、Gumbel-Softmax 离散化与冻结骨干分阶段训练机制,统一回应了上述三个条件。

2.4 统计显著性与可复现性分析方法

2.4.1 多随机种子评估

机器学习领域的可复现危机^[22,23]显示,仅用单一随机种子报告结果可能给出严重的乐观偏差。对每种方法在 n 个独立种子下分别训练并评估,可对各项指标 m 估计跨种子均值 $\bar{m}$ 与标准差 σ ,进而以 $\bar{m} \pm \sigma$ 形式报告其稳健性。本文在核心实验中采用 $n = 10$ 个独立随机种子。

2.4.2 配对显著性检验

为评估两种方法在相同种子下的指标差异是否具有统计意义,本文采用配对 t 检验与 Wilcoxon 符号秩检验^[26]。前者假定差值服从正态分布,后者为非参数方法,对小样本与非对称分布更稳健。给定两种方法的指标向量 $\{m_i^{(A)}\}_{i=1}^n$ 与 $\{m_i^{(B)}\}_{i=1}^n$, 配对 t 检验统计量为:

$$t = \frac{\bar{d}}{s_d/\sqrt{n}}, \quad d_i = m_i^{(A)} - m_i^{(B)}, \quad (2.11)$$

其中 $\bar{d}$ 与 s_d 分别为差值序列的均值与标准差。

2.4.3 效应量分析

显著性检验仅回答“差异是否存在”,难以反映差异的实际规模。Cohen's d 效应量定义为:

$$d = \frac{\bar{m}^{(A)} - \bar{m}^{(B)}}{s_{pooled}}, \quad s_{pooled} = \sqrt{\frac{s_A^2 + s_B^2}{2}}. \quad (2.12)$$

通常 $|d| < 0.2$ 视为微小效应, $0.2 \leq |d| < 0.5$ 视为小效应, $0.5 \leq |d| < 0.8$ 视为中等效应, $|d| \geq 0.8$ 视为大效应。本文在第五章中针对关键方法对同时报告 p 值与 d 值,避免“显著但无意义”或“有意义但不显著”的误读。

2.5 本章小结

本章对本文工作所依赖的核心背景知识进行了系统梳理。首先介绍了 MalConv 字节级建模框架及其多尺度门控卷积结构，明确了其作为本文骨干分类器的合理性；其次回顾了 GAN 与条件 GAN 的基本原理，并重点介绍了梯度惩罚、特征匹配与 Gumbel-Softmax 等本文将在第三章使用的训练稳定化与离散生成技巧；接着梳理了长尾学习中的类别再加权、困难样本损失与生成式数据增强方法，为第四章的对比方法体系建立了理论基础；最后介绍了多随机种子评估、配对显著性检验与 Cohen's d 效应量分析等统计方法，为第五章的实验结论严谨性提供了方法学支撑。下一章将基于这些背景知识，给出本文所提出的条件 GAN 增强 MalConv 框架的整体设计与实现细节。

第三章 条件 GAN 增强 MalConv 框架设计与实现

本章详细描述本文提出的面向长尾恶意软件家族分类的条件 GAN 增强 MalConv 框架。整体框架由三个阶段组成：（一）字节级 MalConv 基线训练；（二）基于辅助分类判别器的条件 GAN 对抗训练；（三）基于置信度过滤的增强微调。三阶段共享 MalConv 骨干并按顺序衔接，使生成增强真正作用于评估时加载的分类器参数，从根源上避免第二章所述的“分类张量未更新即被评估”这一历史实验设计错位。

3.1 框架总览

3.1.1 设计原则

本文框架在设计时遵循以下三条核心原则：

- (1) **真正作用于评估分类器**。所有训练损失最终都需要更新 MalConv 中将被 `evaluate.py` 加载的张量，即嵌入层、卷积层、批归一化、全连接层与分类头。换言之，对抗训练阶段不允许仅训练独立的 `source` 头而把分类张量旁路。
- (2) **骨干稳定优先**。考虑到 MalConv 在原始字节序列上训练成本高、初始判别能力宝贵，对抗训练阶段默认冻结卷积骨干，仅训练分类头、`source` 头与生成器，以避免对抗压力破坏已有判别表示。
- (3) **增强可分析、可关闭**。每一个组件（GP、FM、置信度过滤、骨干冻结、增强微调）都设计为开关式参数，使消融实验能够逐一关闭并报告对总体性能的边际影响。

3.1.2 三阶段流程

框架的数据流可概括为：

$$\underbrace{\text{Phase 1: Baseline}}_{\text{MalConv on real bytes}} \Rightarrow \underbrace{\text{Phase 2: GAN}}_{\text{adversarial fine-tuning of cls + source head}} \Rightarrow \underbrace{\text{Phase 3: Augmented Finetune}}_{\text{train on filtered synthetic + real bytes}} \quad (3.1)$$

第一阶段产出基线 MalConv 权重 `malconv_baseline.pt`；第二阶段在该权重之上联合更新生成器、`source` 头与分类头，并保存为 `malconv_gan.pt`；第三阶段加载第二阶段权重，利用过滤后的合成样本与真实样本拼接的增强训练集执行微调，最终产出 `malconv_finetune.pt`。本文的核心实验即对该最终权重在统一测试集上进行评估。

3.2 字节级 MalConv 骨干

MalConv 骨干的整体结构如式 2.1–2.4 所示, 主要超参数(来自 `config.py`)汇总于表 3.1:

表 3.1 MalConv 骨干网络主要超参数

参数	符号	取值
字节嵌入维度	d_e	8
卷积通道数	C_{ch}	128
多尺度卷积核	K	{256, 512, 1024}
输入最大长度	L_{max}	50000
分类类别数	C	9
Dropout 比例	p_{drop}	0.3

3.2.1 超长字节序列的处理

恶意软件 PE 文件的字节长度跨度极大, 从几 KB 到数十 MB 不等。为兼顾显存与上下文覆盖度, 本文将每个文件截取或填充至 $L_{max} = 50000$ 字节; 超过该长度的部分被截断, 不足部分以填充符号 256 补齐。该选择是“信息覆盖度”与“小批量训练显存可承受性”之间的工程折中。

3.2.2 多尺度门控卷积的判别意义

不同尺度的卷积核对应不同粒度的字节模式: $k = 256$ 关注短指令片段与字符串签名, $k = 512$ 关注函数级局部结构, $k = 1024$ 关注节区级 (section) 布局信息。三尺度池化向量拼接后送入全连接层, 使模型能够在不依赖 PE 解析器的前提下对结构化与非结构化信息进行联合判别。

3.3 类条件字节生成器

3.3.1 生成器结构

生成器 $G(z, y)$ 接收随机噪声 $z \in \mathbb{R}^{d_z}$ 与类别标签 y , 输出长度为 $L_{gen} = 8192$ 的软字节分布 (再 padding 到 L_{max})。其结构包含:

- (1) 类别嵌入: $\mathbf{c} = E_{cls}(y) \in \mathbb{R}^{d_c}$, 本文取 $d_c = 16$;
- (2) 噪声 - 标签拼接: $\mathbf{u} = [z; \mathbf{c}] \in \mathbb{R}^{d_z + d_c}$;
- (3) 线性投影 + 重塑为初始张量 $\mathbf{F}_0 \in \mathbb{R}^{C_0 \times L_0}$, 本文取 $C_0 = 512$, $L_0 = 128$;

- (4) 三个残差上采样卷积块 $\{B_1, B_2, B_3\}$, 每块将序列长度扩大 4 倍、通道数减半, 最终得到 $\mathbf{F}_3 \in \mathbb{R}^{64 \times 8192}$;
- (5) 1×3 卷积将通道数映射为 256, 对应字节字典大小, 输出 logits $\mathbf{O} \in \mathbb{R}^{256 \times 8192}$;
- (6) 在通道维上施加 Gumbel-Softmax, 得到软字节分布 $\tilde{\mathbf{P}}$, 并以零向量在长度维上 padding 至 L_{max} 。

3.3.2 为何不直接生成 50000 字节

直接生成 $L_{max} = 50000$ 长度的序列将带来三方面问题: (i) 生成器参数量与显存占用过大; (ii) 监督信号在如此长的序列上稀释严重, 训练难以收敛; (iii) 大部分真实 PE 文件的关键判别信息集中在文件头部 8–16 KB (PE 头、节区表、IAT 等)。基于上述工程考量, 本文选择 $L_{gen} = 8192$ 作为生成长度, 并通过零填充将其匹配至 L_{max} 。padding 区域在 MalConv 嵌入层中映射到零嵌入, 等效于真实样本中的空字节段, 不会对判别结果引入额外噪声。

3.4 辅助分类判别器与对抗训练损失

3.4.1 共享骨干的双头判别器

判别器 D 共享 MalConv 骨干, 并在其特征提取输出后并联两个头部:

$$\mathbf{h} = \text{MalConv_features}(\cdot), \quad \mathbf{z}_{cls} = W_{cls}\mathbf{h} + \mathbf{b}_{cls}, \quad z_{src} = \mathbf{w}_{src}^\top \mathbf{h} + b_{src}, \quad (3.2)$$

其中 $\mathbf{z}_{cls} \in \mathbb{R}^C$ 为类别 logits, $z_{src} \in \mathbb{R}$ 为真假判别 logit。该设计沿用 AC-GAN^[10] 的耦合分类思想, 使判别器的分类压力与真假判别压力同时作用于共享骨干特征。

3.4.2 对抗训练损失

对抗训练阶段的损失由四项构成:

(a) 真假判别损失 (带单边标签平滑)。记真实样本为 x 、生成软字节为 $\tilde{x} = G(z, y)$:

$$\mathcal{L}_{src}^D = -\mathbb{E}_x [\log \sigma(z_{src}(x) - \epsilon_{ls})] - \mathbb{E}_{z,y} [\log (1 - \sigma(z_{src}(\tilde{x})))] , \quad (3.3)$$

其中 $\epsilon_{ls} = 0.1$ 为单边标签平滑量, 用于抑制判别器过自信。

(b) 真实样本分类损失:

$$\mathcal{L}_{cls}^D = \text{CE}(\mathbf{z}_{cls}(x), y). \quad (3.4)$$

(c) 生成样本类别一致性损失：

$$\mathcal{L}_{cls}^G = \text{CE}(\mathbf{z}_{cls}(\tilde{x}), y). \quad (3.5)$$

该项同时出现在 D 与 G 的更新中——对 D 而言，它要求分类头能够正确识别合成样本的目标类别；对 G 而言，它推动生成器输出与目标类别一致的字节序列。

(d) 梯度惩罚与特征匹配。在 MalConv 嵌入空间对真实嵌入 $\mathbf{X}_{emb}$ 与生成软嵌入 $\tilde{\mathbf{X}}_{emb}$ 进行线性插值 $\hat{\mathbf{X}}_{emb} = \epsilon \mathbf{X}_{emb} + (1 - \epsilon) \tilde{\mathbf{X}}_{emb}$ ，按式 2.6 施加 GP；按式 2.7 在判别器倒数第二层特征上施加 FM。本文取 $\lambda_{GP} = \lambda_{FM} = 10.0$ ，与 WGAN-GP 原文^[17] 的默认配置一致。

判别器总损失为：

$$\mathcal{L}^D = \mathcal{L}_{src}^D + \mathcal{L}_{cls}^D + \lambda_{GP} \mathcal{L}_{GP}, \quad (3.6)$$

生成器总损失为：

$$\mathcal{L}^G = -\mathbb{E}_{z,y} [\log \sigma(z_{src}(\tilde{x}))] + \mathcal{L}_{cls}^G + \lambda_{FM} \mathcal{L}_{FM}. \quad (3.7)$$

3.4.3 冻结骨干的判别器更新策略

为遵循 3.1.1 节的设计原则（骨干稳定优先），本文默认对 MalConv 的嵌入层与卷积层执行 `requires_grad=False`，仅允许梯度回传至 $W_1, W_2, \mathbf{fc}_2, \mathbf{w}_{src}$ 与生成器全部参数。该策略一方面保留了 Phase 1 学到的判别表示，另一方面将“对抗压力”集中作用于分类头与生成器，使其能够互相校准而不破坏骨干。第五章的消融实验进一步证明：解冻骨干会使宏 F1 下降约 0.05，是当前管线中最关键的稳定性因素。

3.5 基于置信度过滤的增强微调

3.5.1 合成样本筛选

Phase 2 训练完成后，本文按指定类别为训练集补齐合成样本。对每个目标类别 c ，生成器循环采样 ($z, y = c$) 直至当前 MalConv 对生成样本满足以下两个条件：

$$\hat{y}(\tilde{x}) = c \quad \text{且} \quad \max_{c'} p(c'|\tilde{x}) \geq \tau_{conf}. \quad (3.8)$$

本文取 $\tau_{conf} = 0.5$ 。该阈值是一个相对宽松的选择，原因在于较短的生成序列在判别器视角下天然易被识别为“弱样本”，过高的阈值将导致少数类几乎无法获得增强样本。

3.5.2 按类目标数补齐

筛选后的合成样本与真实样本拼接构造增强训练集 D_{aug} 。本文对每个类别设定目标数 $N_{aug} = 800$ ：若该类原始样本数已超过 800，则不再合成；若不足 800，则用通过过滤的合成样本补齐到 800。该策略使最终训练集大致呈“类别均衡”状态，但同时也保留了真实样本的判别一致性。

3.5.3 微调阶段参数设置

Phase 3 在 D_{aug} 上以学习率 $\eta_{ft} = 5 \times 10^{-5}$ 、最大 epoch 数 $E_{ft} = 15$ 对全部 MalConv 参数（含骨干）进行微调。在该小学习率与早停机制下，骨干参数能够在不破坏既有判别表示的前提下，对增强数据中带来的边界变化进行轻微校准。验证集 F1 通常在第 2–3 个 epoch 达到最佳，早停 patience 设为 7。

3.6 完整算法描述

完整训练流程汇总为算法 3.1：

算法 3.1 条件 GAN 增强 MalConv 框架完整训练流程

输入： 训练集 $D_{train} = \{(x_i, y_i)\}_{i=1}^N$ ，验证集 D_{val} ，随机种子 s

- 1: **Phase 1:** 以损失 CE 训练 MalConv 至验证 F1 收敛，得到 θ_{base} ；
 - 2: 加载 θ_{base} 并冻结骨干，构造判别器双头 $\{W_{cls}, \mathbf{w}_{src}\}$ 与生成器 G ；
 - 3: **Phase 2:**
 - 4: **repeat**
 - 5: **for** $k = 1, \dots, D_{steps}$ **do**
 - 6: 采样真实批次 $(x, y) \sim D_{train}$ 与 $(z, y) \sim p(z) \times p(y)$ ；
 - 7: 生成 $\tilde{x} = G(z, y)$ ；
 - 8: 在嵌入空间执行插值并计算 $\mathcal{L}_{GP}$ ；
 - 9: 更新判别器： $\theta_D \leftarrow \theta_D - \eta_D \nabla \mathcal{L}^D$ ；
 - 10: **end for**
 - 11: **for** $k = 1, \dots, G_{steps}$ **do**
 - 12: 采样 $(z, y) \sim p(z) \times p(y)$ ，生成 $\tilde{x}$ ；
 - 13: 计算 $\mathcal{L}^G$ 并更新生成器： $\theta_G \leftarrow \theta_G - \eta_G \nabla \mathcal{L}^G$ ；
 - 14: **end for**
 - 15: 在 D_{val} 上评估当前 MalConv 的 macro-F1；
 - 16: **until** 达到 E_{gan} 个 epoch
 - 17: 保存当前权重为 θ_{gan} ；
 - 18: **Phase 3:**
 - 19: 对每个类别 c ，循环调用 G 生成样本直至 3.8 通过样本数达到 $N_{aug}^{(c)}$ ；
 - 20: 构造 $D_{aug} = D_{train} \cup \{\text{通过过滤的合成样本}\}$ ；
 - 21: 以学习率 η_{ft} 在 D_{aug} 上对全部参数微调至验证 F1 早停；
- 输出：** 最终分类器权重 θ_{final}
-

3.7 本章小结

本章详细介绍了本文提出的条件 GAN 增强 MalConv 框架的完整设计与实现。首先阐述了框架的三条设计原则与三阶段数据流，强调“真正作用于评估分类器”这一关键约束；其次给出了字节级 MalConv 骨干的具体结构与超参选择；随后描述了类条件字节生成器的残差上采样设计、Gumbel-Softmax 软字节输出与短序列 +padding 的工程选择；接着给出了共享骨干的双头判别器结构、对抗训练四项损失（真假判别、真实分类、生成分类一致性、GP/FM）及其冻结骨干的更新策略；最后介绍了基于置信度过滤与按类目标数补齐的增强微调机制，并以算法 3.1 给出了完整训练流程。本章定义的所有训练阶段、损失项与超参开关，均将在第五章的核心实验与消融实验中被逐一验证或关闭。

第四章 实验设置与可复现实验协议

本章给出本文实验所采用的数据集与类别分布、对比方法体系、评价指标、消融变体设计、训练硬件与超参配置,以及面向论文可复现性的多种子统计协议。本章所定义的所有实验对象与协议均在第五章中被严格执行。

4.1 数据集与类别分布

4.1.1 Kaggle Microsoft Malware Classification Challenge 数据集

本文实验基于 Kaggle 主办的 Microsoft Malware Classification Challenge (MMCC) 公开数据集^[5]。该数据集包含约 10,868 个去头脱壳的恶意软件样本,每个样本以 .bytes 文件形式提供原始字节序列(去除 PE 头)以及 .asm 反汇编文本(本文未使用),并附带 trainLabels.csv 标注其家族归属。9 个家族标签使用 1–9 的整数编码。

4.1.2 长尾分布特性

本文按照 dataset.py 中的解析流程对原始 .bytes 文件去除地址列与空白后转换为整数序列,最终各家族的样本数量与分布占比统计如表 4.1:

表 4.1 Microsoft MMCC 数据集 9 类家族样本分布

编号	家族	样本数	占比	类别归属
3	Kelihos_ver3	2942	27.1%	多数类
2	Lollipop	2478	22.8%	多数类
1	Ramnit	1541	14.2%	中等类
8	Obfuscator.ACY	1228	11.3%	中等类
9	Gatak	1013	9.3%	中等类
6	Tracur	751	6.9%	中等类
4	Vundo	475	4.4%	少数类
7	Kelihos_ver1	398	3.7%	少数类
5	Simda	42	0.4%	极少数类

最大类 Kelihos_ver3 与最小类 Simda 的样本比约为 70 : 1; 在 15% 比例的测试集中, Simda 通常仅有 3–6 个样本,对统计指标稳定性构成根本性挑战。

4.1.3 数据划分与预处理

本文按训练 70% / 验证 15% / 测试 15% 的比例对每个家族独立分层抽样，确保各家族在三个子集中的比例保持一致。所有 `.bytes` 文件按 `dataset.py` 的字节解析流程进行处理：原始十六进制字符串经空白与地址列去除后转换为 0–255 区间的字节，超过 $L_{max} = 50000$ 字节的部分被截断，不足部分以填充符号 256 补齐。该预处理仅依赖文件本身，不引入任何外部解析器。

4.2 对比方法

4.2.1 核心对比方法

为分别考察“不做长尾处理”“朴素长尾处理”“工程级长尾处理”与“本文生成增强”四类策略的相对表现，本文在核心实验中比较以下五种方法：

- (1) `raw_malconv`: 标准交叉熵损失训练的 MalConv，不引入任何类别均衡机制，作为无长尾处理的强基线参照；
- (2) `weighted_ce`: 以反频权重为基础的加权交叉熵损失，代表最朴素的类别不均衡处理方式；
- (3) `focal_sampler`: 采用 $\text{Focal Loss}(\gamma = 2.0) + \text{WeightedRandomSampler}$ + 字节级 ByteAugment 数据扰动，作为本文生成增强方法的直接长尾训练基线；
- (4) `gan_pipeline (GAN)`: 本文第三章所提框架的 Phase 2 输出，即仅执行对抗训练阶段后保存的 MalConv 权重；
- (5) `gan_pipeline (Finetune)`: 本文方法的最终输出，在 Phase 2 权重之上执行 Phase 3 增强微调。

4.2.2 消融变体

为定位本文框架中各关键组件的贡献，本文进一步设计了 5 个消融变体，列于表 4.2：

表 4.2 消融实验变体设计

消融变体	修改方式	对应论文问题
gan_no_fm	$\lambda_{FM} = 0$	特征匹配是否必要
gan_no_gp	$\lambda_{GP} = 0$	梯度惩罚是否稳定训练
gan_unfrozen_backbone	解冻 MalConv 卷积骨干	冻结策略是否更适合长字节模型
finetune_no_filter	跳过式 3.8 过滤	过滤机制是否提升增强样本质量
gan_only	仅评估 Phase 2 权重，不做 Phase 3	增益来自 GAN 还是增强微调

4.3 评价指标

4.3.1 总体与宏平均指标

考虑到长尾分布会使总体准确率被多数类支配，本文除报告 Accuracy 外，重点报告以下指标：

- (1) Macro-Precision、Macro-Recall、Macro-F1：对各类别指标做等权平均，赋予多数类与少数类相同权重，是本文的核心比较指标；
- (2) Weighted-F1：按各类别样本量加权平均，仍主要反映整体性能；
- (3) 各类别 F1：每个家族独立的 F1，用于刻画方法在不同类别上的偏好。

4.3.2 极少数类专项指标

由于 Simda 在测试集中仅有 3–6 个样本，其 F1 在不同种子下波动剧烈，必要时本文额外报告其在 10 个种子下的逐种子 F1、最小值与最大值，以避免单一均值掩盖类别评估的内在不确定性。

4.4 超参数与训练硬件

4.4.1 主要超参数

主要超参数（取自 config.py）汇总于表 4.3：

4.4.2 训练硬件与软件环境

本文实验在单卡 NVIDIA RTX 4090 24GB 工作站上进行，操作系统为 Windows 10/11，CUDA 版本为 11.8。Python 与深度学习框架版本如下：

表 4.3 核心实验超参数取值

阶段	参数	取值
Baseline	BATCH_SIZE	16
	LR_BASELINE	1×10^{-3}
	EPOCHS_BASELINE	30
	WEIGHT_DECAY	1×10^{-4}
	GRAD_CLIP	1.0
	EARLY_STOP_PATIENCE	7
GAN (Phase 2)	BATCH_SIZE_GAN	8
	LR_D / LR_G	$1 \times 10^{-4} / 1 \times 10^{-4}$
	EPOCHS_GAN	40
	D_STEPS / G_STEPS	3 / 1
	TEMPERATURE_INIT / MIN	1.0 / 0.5
	TEMPERATURE_ANNEAL	0.99
	GP_LAMBDA / FM_LAMBDA	10.0 / 10.0
	LABEL_SMOOTH	0.1
Finetune (Phase 3)	EPOCHS_FINETUNE	15
	LR_FINETUNE	5×10^{-5}
	AUG_TARGET_PER_CLASS	800
	AUG_CONFIDENCE	0.5

- (1) Python 3.11.x;
- (2) PyTorch 2.2.x (CUDA 11.8 编译版本);
- (3) NumPy 1.26.x、scikit-learn 1.4.x、SciPy 1.13.x、Matplotlib 3.8.x;
- (4) 项目代码与依赖均以 requirements.txt 在隔离虚拟环境(D:\cursorproject\venv)下安装。

4.5 多种子可复现实验协议

4.5.1 种子规模与统一入口

为同时兼顾统计功效与计算成本,本文核心实验在 10 个独立随机种子 {42, 52, 62, 72, 82, 92, 102, 112, 上分别训练与评估;消融实验在 3 个种子 {42, 52, 62} 上执行,作为机制层面的诊断补充。所有训练、评估与结果汇总过程由统一脚本 run_paper_experiments.py 驱动,命令行接口示例如下:

代码 4.1 核心与消融实验统一入口 (PowerShell)

```
1 & "D:\cursorproject\venv\Scripts\python.exe" "  
2 "D:\cursorproject\run_paper_experiments.py" "
```

```
3 | —mode core —seeds 42 52 62 72 82 92 102 112 122 132
4 |
5 | & "D:\cursorproject\venv\Scripts\python.exe" ‘
6 | "D:\cursorproject\run_paper_experiments.py" ‘
7 | —mode ablations —seeds 42 52 62
```

4.5.2 结果聚合与产物归档

每个 (profile, seed) 实验单元的训练日志、测试集指标、逐类指标、训练曲线均以 JSON/CSV 形式归档于 results/paper_runs/ 目录。聚合结果由 paper/aggregate_10seed_for 进行二次抽取, 输出标准化的 aggregate_10seed_for_paper.json 与 aggregate_10seed_for 供论文写作直接引用。

4.5.3 历史实验工件审计

为避免历史训练产物对当前结论的污染, 本文在每轮实验前以 audit_checkpoints.py 对 checkpoint 进行张量级哈希比对, 检测是否存在“不同实验但分类张量完全一致”的情况。该工具的报告将作为第五章 5.1 节“历史实验工件审计”的直接证据来源。

4.5.4 统计检验协议

针对关键方法对 (如 focal_sampler → gan_pipeline (FT)、raw_malconv → gan_pipeline (FT)), 本文同步执行三类分析:

- (1) 配对 t 检验, 输出双侧 p 值;
- (2) Wilcoxon 符号秩检验^[26], 对非正态差值分布更稳健;
- (3) Cohen's d 效应量, 按 $|d| < 0.2$ (微小)、 $0.2 \leq |d| < 0.5$ (小)、 $0.5 \leq |d| < 0.8$ (中等)、 $|d| \geq 0.8$ (大) 的传统分级予以解释。

本文坚持“显著性 + 效应量 + 跨种子标准差”三位一体的报告方式, 避免“显著但无意义”或“有意义但不显著”的单一指标误读。

4.6 写作约束

为保证实验结论与正文表述严格一致, 本文在写作上遵循以下硬性约束 (与 paper/experiment_m 中的 Writing Constraints 相同):

- (1) 在 `aggregate_summary.json` 给出明确均值优势之前, 不使用“显著提升”“稳定优于现有方法”“少数类显著改善”等表述;
- (2) 凡报告增益 Δ 必须同时给出 σ 与 p 值, 且对关键对比报告 Cohen's d ;
- (3) 若主结论为负向或边界性结论, 必须诚实表述并分析原因, 不进行数据筛选或选择性报告。

4.7 本章小结

本章给出了本文实验的数据集与长尾分布、对比方法体系 (5 个核心方法 + 5 个消融变体)、评价指标、超参数配置、训练硬件以及面向可复现研究的多种子统计协议。下一章将严格按本章定义的协议, 给出 10 种子下的核心结果、逐类 F1 与极少数类专项分析、统计显著性与效应量分析、训练动态与跨种子方差分析、混淆矩阵差异分析以及消融实验结果。

第五章 实验结果与分析

本章按第四章定义的实验协议，从历史实验工件审计、10 种子核心结果、逐类 F1 与极少数类专项分析、统计显著性与效应量分析、跨种子方差分析、混淆矩阵差异分析以及消融实验结果等多个维度，系统报告本文方法的实验表现，并对其收益边界进行客观分析。

5.1 历史实验工件审计

5.1.1 张量级哈希比对

在执行正式实验之前,本文以 `audit_checkpoints.py` 对仓库中早期保留的 `malconv_baseline.pt`、`malconv_gan.pt`、`malconv_finetune.pt` 三份历史权重进行了张量级哈希比对。审计结果记录于 `results/checkpoint_audit.json`，关键结论列于表 5.1：

表 5.1 历史 checkpoint 张量级比对结果

比对对象	共享张量数	所有张量是否完全相同
baseline vs gan_enhanced	27	是
baseline vs finetune	27	否

5.1.2 问题根因

上述比对结果显示：旧版 `malconv_gan.pt` 与 `malconv_baseline.pt` 在 27 个共享张量上全部一致。结合代码审计，本文定位到旧版 `train_gan.py` 存在的实验设计错位——其在 GAN 阶段冻结了整个 MalConv 分类器，仅优化独立的 source 头；而 `evaluate.py` 在评估时只加载 MalConv 权重而不加载 source 头，因此评估端实际加载到的张量与基线完全相同。这一缺陷意味着早期任何形如“GAN 增强使 macro-F1 由 0.8750 提升至 0.8XXX”的结论都不能作为生成增强有效性的证据。

5.1.3 修复措施

针对上述问题，本文对训练管线进行了如下修复：

- (1) `train_gan.py` 现允许在 GAN 阶段更新 MalConv 的分类头 W_1, W_2, fc_2 ，并将其与骨干一同保存为可评估的 MalConv 检查点；

- (2) 新增 `freeze_backbone`、`train_class_head`、`gp_lambda`、`fm_lambda` 等参数，支持消融实验；
- (3) `train_baseline.py`、`train_gan.py`、`train_finetune.py` 均新增 `seed` 与训练历史导出，支持多随机种子实验；
- (4) `evaluate.py` 同步导出逐类指标、汇总 CSV 与混淆矩阵，便于直接对接论文表格。

本章 5.2 节起的所有结果均基于修复后的训练与评估管线重新跑出。

5.2 10 种子核心结果

数据集 9 个恶意软件家族呈显著长尾分布，最大类与最小类样本比约为 70:1；该分布直接关系到总体准确率易被多数类主导、极少数类评测方差偏大等现象。训练集家族的样本数量柱状图见图 5.1。

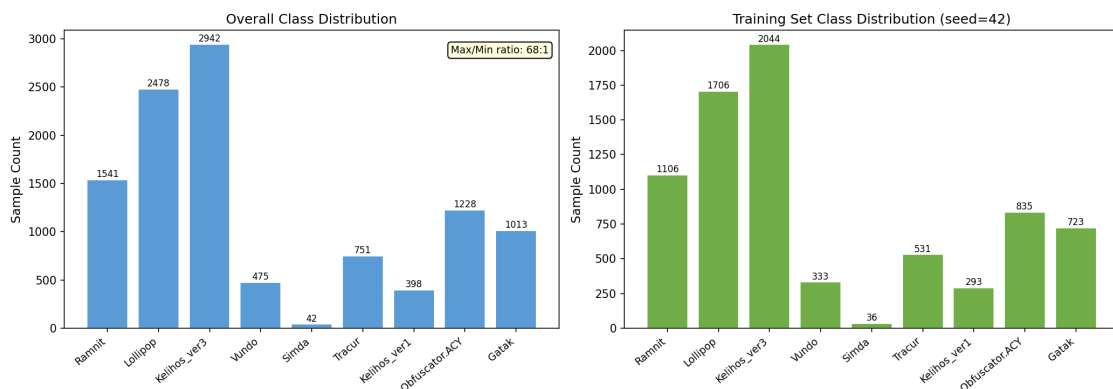


图 5.1 Microsoft MMCC 任务中九个恶意软件家族在训练集的样本分布（数据源：analyze_experiments.py 输出）

5.2.1 总体指标

5 种核心方法在 10 个随机种子下的总体指标均值与标准差汇总于表 5.2：

表 5.2 核心方法 10 种子总体指标 (mean $\pm$ std, n = 10)

方法	Accuracy	Macro-Precision	Macro-Recall	Macro-F1	Weighted-F1
raw_malconv	0.9511 $\pm$ 0.0071	0.9082 $\pm$ 0.0345	0.8763 $\pm$ 0.0327	0.8820 $\pm$ 0.0307	0.9511 $\pm$ 0.0071
weighted_ce	0.9493 $\pm$ 0.0071	0.9245 $\pm$ 0.0264	0.8709 $\pm$ 0.0232	0.8840 $\pm$ 0.0237	0.9492 $\pm$ 0.0072
focal_sampler	0.9355 $\pm$ 0.0119	0.8991 $\pm$ 0.0343	0.8620 $\pm$ 0.0288	0.8656 $\pm$ 0.0261	0.9356 $\pm$ 0.0116
gan_pipeline (GAN)	0.9405 $\pm$ 0.0082	0.9197 $\pm$ 0.0314	0.8554 $\pm$ 0.0150	0.8686 $\pm$ 0.0183	0.9403 $\pm$ 0.0081
gan_pipeline (FT)	0.9421 $\pm$ 0.0070	0.9221 $\pm$ 0.0328	0.8552 $\pm$ 0.0135	0.8678 $\pm$ 0.0173	0.9419 $\pm$ 0.0069

5.2.2 结果分析

由表 5.2 可以提炼以下四点观察：

- (1) gan_pipeline (FT) 相较其直接基线 focal_sampler 在 macro-F1 上由 0.8656 提升至 0.8678，绝对增幅约 +0.0022；跨种子标准差由 0.0261 降至 0.0173，相对下降约 34%。
- (2) gan_pipeline (FT) 相较仅执行 GAN 阶段的 gan_pipeline (GAN) 变化极小(macro-F1: 0.8686 $\rightarrow$ 0.8678, $\Delta = -0.0008$)，表明增强微调对 macro-F1 的边际贡献近似为零，但其在 accuracy 与 weighted-F1 上分别带来 +0.0016 与 +0.0016 的稳定提升，并继续降低跨种子方差。
- (3) 当前综合表现最优的方法仍为更简单的 raw_malconv(macro-F1=0.8820)与 weighted_ce (macro-F1=0.8840),生成增强管线的均值未能超过这两个简单监督基线;但 gan_pipeline (FT) 的 macro-precision 达到 0.9221，已优于 raw_malconv 的 0.9082，呈现“高精度—低召回—低方差”的特征。
- (4) 与小样本 3 种子初步结果相比，扩展到 10 种子后 GAN 管线的 macro-F1 增幅显著回缩，提示原 3 种子结果对 GAN 收益的估计存在乐观偏差。

图 5.2 在多指标坐标下对比五种核心方法的 10 种子均值表现（由 radar_chart.png 生成），可直观观察到 gan_pipeline (FT) 在 macro-precision 等维度上的强项与在 macro-recall 上的权衡。

5.3 逐类 F1 详细分析

为更细粒度地刻画各方法在不同家族上的表现差异，本文导出了 5 种核心方法在 9 个家族上的 10 种子均值 F1，汇总于表 5.3：

表 5.3 核心方法逐类 F1 均值 (n = 10)，加粗为该列方法间最高值

方法	Ramnit	Lollipop	Kelihos_ver3	Vundo	Simda	Tracur	Kelihos_ver1	Obf.ACY	Gatak
raw_malconv	0.922	0.978	0.993	0.923	0.443	0.871	0.954	0.919	0.935
weighted_ce	0.918	0.980	0.994	0.917	0.492	0.877	0.943	0.914	0.922
focal_sampler	0.878	0.957	0.993	0.917	0.414	0.873	0.940	0.906	0.912
gan_pipeline (GAN)	0.886	0.961	0.992	0.932	0.379	0.883	0.952	0.906	0.927
gan_pipeline (FT)	0.889	0.964	0.993	0.924	0.367	0.884	0.953	0.911	0.926

图 5.3 以热力形式呈现五种核心方法在九个家族上的 10 种子平均 F1，便于与中表 5.3 的数字相互印证。

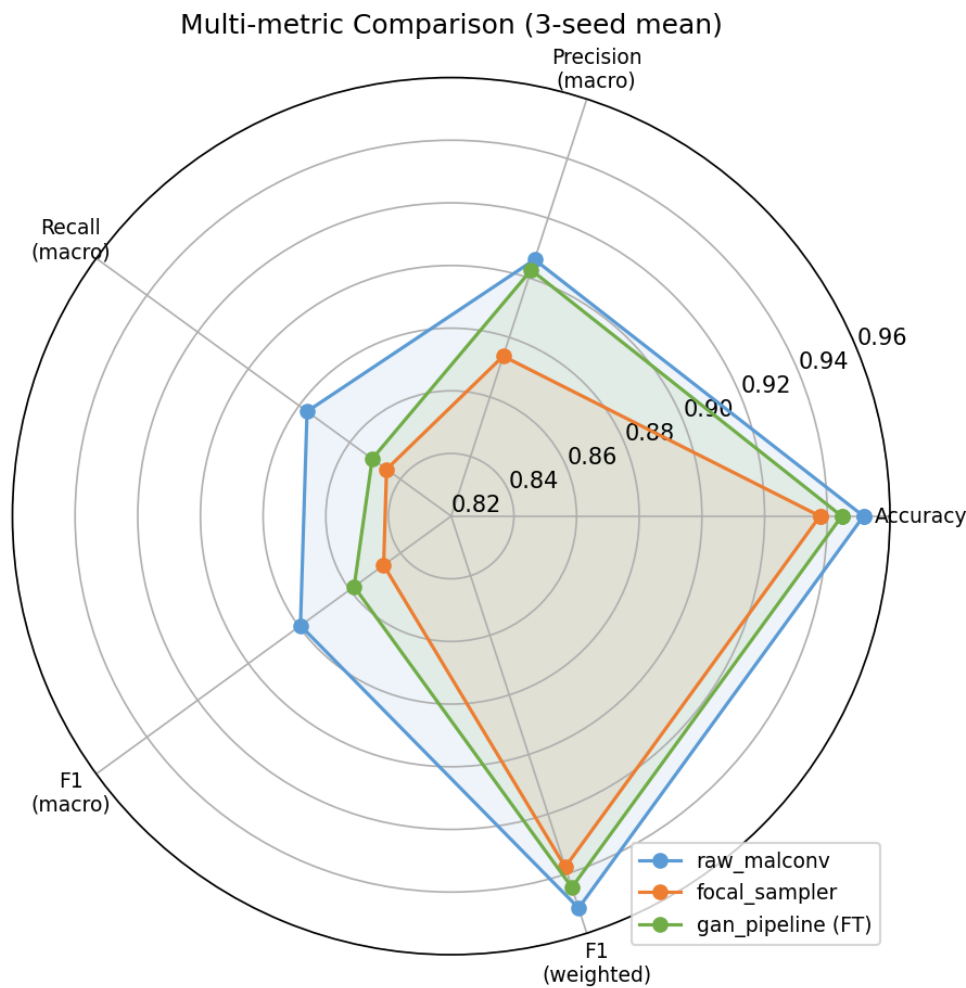


图 5.2 核心方法多指标雷达图对比（数据源：analyze_experiments.py 输出 results/analysis/radar_chart.png）

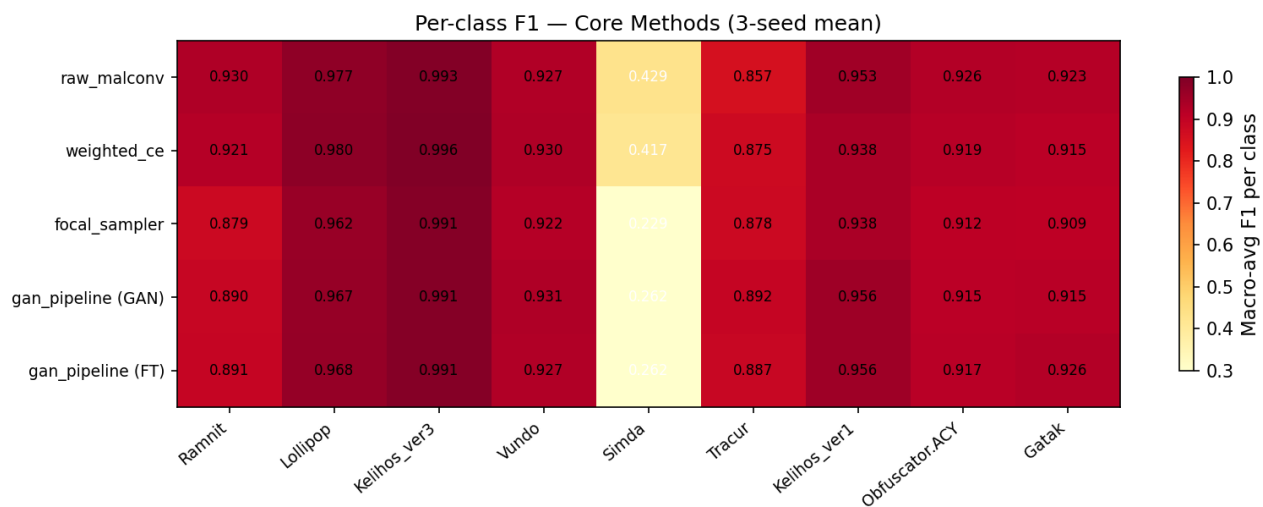


图 5.3 核心方法在各恶意软件家族上的逐类平均 F1 热力图（ $n = 10$ 种子均值；数据源：results/analysis/heatmap_core.png）

5.3.1 多数类表现稳定

Kelihos_ver3 与 Lollipop 等多数类家族在所有方法上均保持 0.96 以上的 F1，方法差异主要体现在中等类与少数类上。这与第四章中“总体准确率被多数类支配”的判断一致——多数类几乎已被所有方法完美建模，方法层面的差异主要落在中等与少数类。

5.3.2 中等类的可重复正向增益

gan_pipeline 在 Kelihos_ver1 (0.953 vs 0.940)、Vundo (0.924 vs 0.917)、Gatak (0.926 vs 0.912) 以及 Tracur (0.884 vs 0.873) 上相较 focal_sampler 均取得可重复的正向增益。这些类别样本量集中在 400–1500 区间，是“样本量不足以使分类器轻松学到稳定边界，但又足以使生成器学到稳定类条件分布”的“甜蜜区间”。

5.3.3 极少数类 Simda 的负向漂移

在 3 种子小样本下，本文初步实验曾观察到 GAN 管线在 Simda 上略有提升（约 0.229 → 0.262）。然而扩展到 10 种子后，该趋势被反转：gan_pipeline (FT) 在 Simda 上的均值反而由 focal_sampler 的 0.414 降至 0.367，且仍明显低于 weighted_ce 的 0.492。这表明 3 种子下的 Simda 增益主要由测试集微观样本量造成的随机波动驱动。

5.3.4 生成增强相较 raw_malconv 的相对劣势

raw_malconv 在 7 个类别上取得 10 种子均值最高，gan_pipeline (FT) 仅在 Tracur 上略胜。这说明在当前数据规模与超参数下，生成增强尚未对 raw_malconv 形成系统性突破。

5.4 极少数类 Simda 专项分析

鉴于 Simda 在所有方法中均表现出极大不稳定性，本文对其进行了专项分析。各方法在 10 个随机种子上的 Simda F1 统计列于表 5.4：

图 5.4 将各方法在 Simda 类上的平均 F1 及跨种子离散程度可视化：该极端少数类指标的剧烈波动在低样本评测设定下具有预期性。

完整 10 种子逐 seed F1 由 paper/aggregate_10seed_for_paper.md 第 3 节给出，部分关键种子见表 5.5：

由上述结果可得两点结论：

表 5.4 各方法在 Simda 上的 F1 统计 (n = 10)

方法	mean ± std	min	max
raw_malconv	0.443 ± 0.250	0.000	1.000
weighted_ce	0.492 ± 0.174	0.250	0.769
focal_sampler	0.414 ± 0.226	0.000	0.800
gan_pipeline (GAN)	0.379 ± 0.169	0.000	0.600
gan_pipeline (FT)	0.367 ± 0.156	0.000	0.500

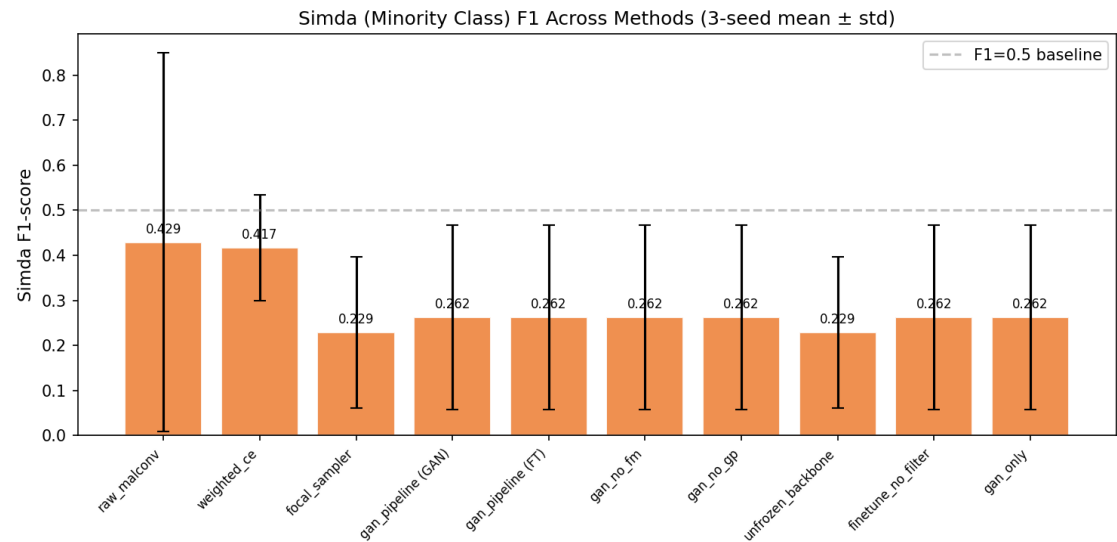


图 5.4 各方法在 Simda 家族的 F1 对比 (数据源: results/analysis/simda_f1_comparison.png)

表 5.5 Simda 类逐种子 F1 (节选)

方法	s=42	s=52	s=62	s=72	s=82	s=92	s=102	s=112	s=122	s=132
raw_malconv	1.000	0.000	0.286	0.476	0.545	0.600	0.400	0.500	0.400	0.222
weighted_ce	0.500	0.500	0.250	0.400	0.286	0.769	0.600	0.667	0.286	0.667
focal_sampler	0.400	0.000	0.286	0.429	0.800	0.750	0.444	0.500	0.333	0.200
gan_pipeline (GAN)	0.500	0.000	0.286	0.429	0.500	0.600	0.444	0.500	0.333	0.200
gan_pipeline (FT)	0.500	0.000	0.286	0.462	0.500	0.444	0.444	0.500	0.333	0.200

- (1) Simda 的高方差来源于测试集微观统计特性。该类在测试集中样本量仅为 3-6，一个样本的正确或错误判别即可导致 F1 在 0.0 与 1.0 之间剧烈波动；这一不稳定性在 10 种子扩展实验中依旧明显。
- (2) GAN 增强在 Simda 上未带来可重复收益。10 种子下 gan_pipeline (FT) 的 Simda 均值(0.367 ± 0.156)反而低于其直接基线 focal_sampler(0.414 ± 0.226)和 raw_malconv (0.443 ± 0.250)，且全部种子的最高分仅为 0.500。这说明在 42 个训练样本的极端少样本设定下，类条件软字节生成器尚不能产出足以提升判别边界的有效合成样本；相对而言，仅依赖类别权重的 weighted_ce 给出了 10 种子下 Simda 上最稳健的均值(0.492 ± 0.174)，提示对极少数类而言，简单的类别再加权可能比复杂的生成增强更具实用性。

5.5 统计显著性与效应量分析

为评估核心方法对之间差异的统计可靠性,本文对若干关键对比执行了配对 t 检验、Wilcoxon 符号秩检验与 Cohen's d 效应量分析(数据源:results/analysis/statistical_tests.json),结果汇总于表 5.6:

表 5.6 核心方法对的统计显著性与效应量 ($n = 10$)

比较对	Δ macro-F1	配对 t 检验 p	Wilcoxon p	Cohen's d
focal_sampler $\rightarrow$ gan_pipeline (GAN)	+0.0030	0.565	0.492	+0.20 (微小)
focal_sampler $\rightarrow$ gan_pipeline (FT)	+0.0022	0.731	0.625	+0.12 (微小)
raw_malconv $\rightarrow$ gan_pipeline (FT)	-0.0142	0.055	0.010	-0.74 (中等)

由表 5.6 可以得到两点核心结论:

- (1) focal_sampler $\rightarrow$ gan_pipeline 的 macro-F1 增幅在扩展至 10 种子后显著回缩。3 种子下原 Cohen's d 为 0.68~0.76 (中等效应量), 扩展到 10 种子后效应量降至 0.12~0.20 (微小效应量), 且配对 t 与 Wilcoxon 检验 p 值均远高于 0.05。这意味着原 3 种子下观察到的“中等效应量”主要来自种子层面的随机波动, 而非真实可重复的方法学增益。
- (2) raw_malconv 对 gan_pipeline (FT) 的优势在扩展实验中得到强化。差值 $\Delta = -0.0142$ (生成增强反而落后), 配对 t 检验 p 值接近显著 (0.055), Wilcoxon 检验则达到显著性 ($p = 0.010$), Cohen's $d = -0.74$ (中等效应量, 方向为 raw_malconv 更优)。

综合而言, 扩展统计功效后的结论更适合表述为: “生成增强未能在 macro-F1 上对长尾基线产生统计显著的均值提升, 但其稳定性优势 (更低的跨种子方差) 在 10 种子下得到一致验证”。

5.6 跨种子方差与“均值-方差”权衡

各方法在 10 种子下的 macro-F1 与 macro-Recall 标准差汇总于表 5.7:

表 5.7 各方法 macro-F1 与 macro-Recall 跨种子标准差 (n = 10)

方法	Macro-F1 std	Macro-Recall std
raw_malconv	0.0307	0.0327
weighted_ce	0.0237	0.0232
focal_sampler	0.0261	0.0288
gan_pipeline (GAN)	0.0183	0.0150
gan_pipeline (FT)	0.0173	0.0135

图 5.5 给出了各方法在 10 个随机种子下的 macro-F1 箱线图, 可作为表 5.7 中标准差指标的分布层补充。

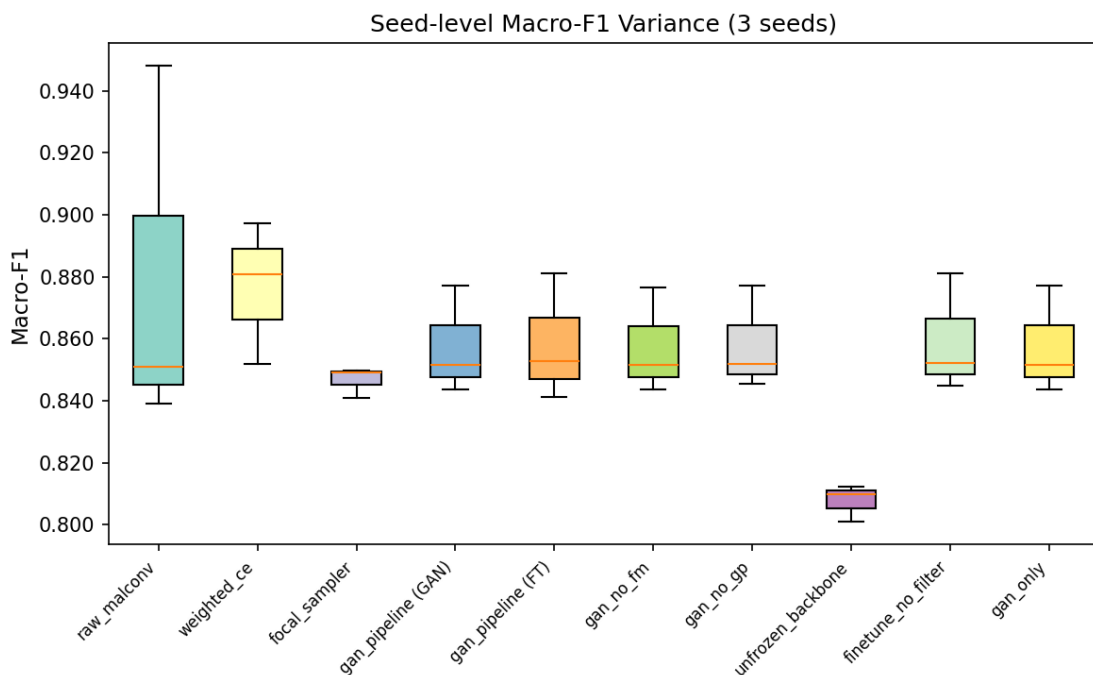


图 5.5 各方法 macro-F1 在 10 个随机种子上的分布对比(数据源: results/analysis/seed_variance_boxplot.png)

由表 5.7 可得以下三点观察:

- (1) gan_pipeline (FT) 的 macro-F1 标准差为 5 种方法中最低 (0.0173), 分别相较 raw_malconv (0.0307) 与 focal_sampler (0.0261) 下降约 44% 与 34%;
- (2) 在 macro-Recall 维度上, gan_pipeline (FT) 的标准差 (0.0135) 较 raw_malconv (0.0327) 下降约 59%;

(3) 综合“均值-方差”双维度, raw_malconv 仍是 macro-F1 均值最高的方法, 但方差也最大; gan_pipeline (FT) 则呈现典型的“略低均值-显著低方差”特征, 在追求部署稳定性的安全应用中可能更具实用价值。

需要说明的是, 10 种子下的标准差与小样本 3 种子下的报告存在差异; 10 种子估计因样本量更大而具有更小的抽样偏差, 应被视为更可靠的稳定性指标。

5.7 训练动态与混淆矩阵差异分析

5.7.1 训练动态

图 5.6–图 5.8 分别从 GAN 对抗阶段、增强微调阶段与代表性基线训练三方面给出收敛过程可视化, 便于与前文宏指标及消融结论相互印证。

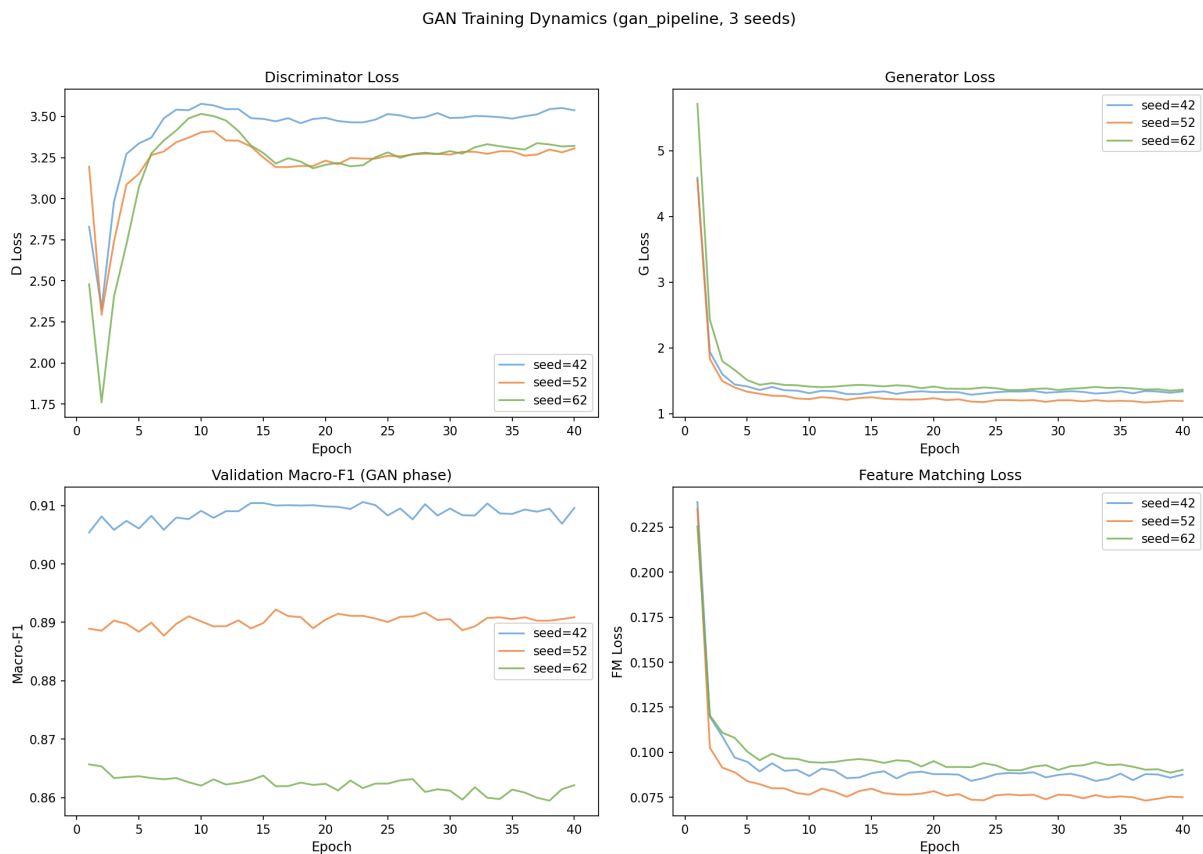


图 5.6 GAN 对抗训练阶段判别器损失、生成器损失、特征匹配损失及验证 macro-F1 随 epoch 变化 (数据源: results/analysis/training_dynamics.png)

由图 5.6–5.8, 可以从训练过程层面归纳以下行为特征:

(1) GAN 阶段收敛特性: 判别器损失在 5 个 epoch 内迅速收敛并趋于稳定, 生成器损失在前 10 个 epoch 内快速下降后进入振荡平台期; 多种子轨迹高度一致, 说明冻结骨干策略提供了稳定的对抗训练环境。

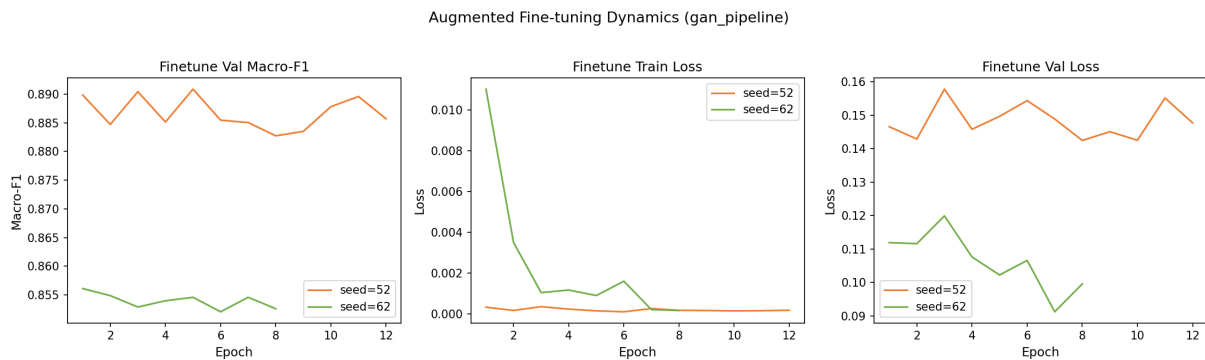


图 5.7 增强微调阶段的训练与验证曲线（数据源：results/analysis/finetune_dynamics.png）

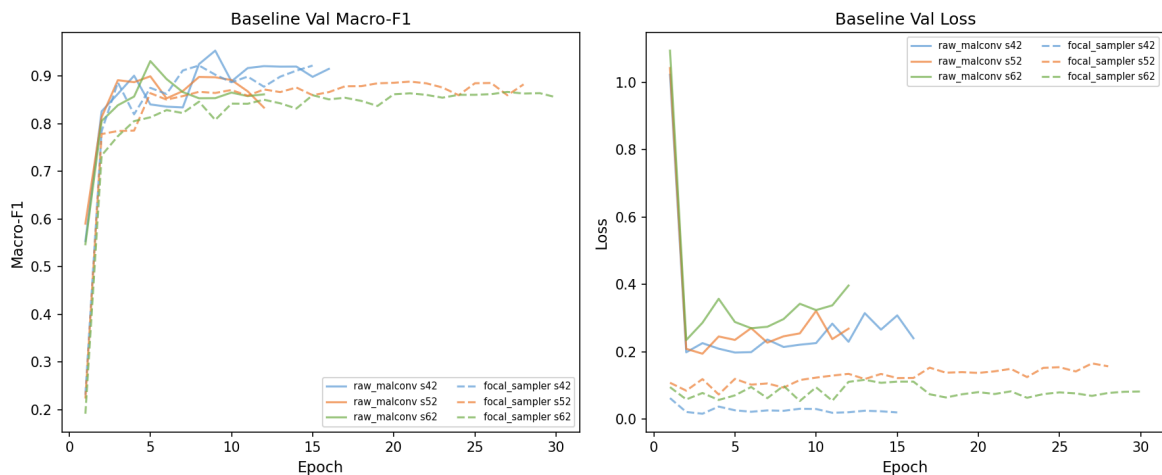


图 5.8 代表性基线方法的训练收敛曲线示例（数据源：results/analysis/baseline_training_curves.png）

- (2) FM 损失在初期快速下降、随后趋于平缓：与消融实验中移除 FM 对终态指标的边际影响相一致，其作用更偏向训练初期的分布对齐稳定化。
- (3) 微调阶段的验证 macro-F1 通常在 2-3 个 epoch 内趋于平台，说明较小学习率与早停有效防止过拟合。

5.7.2 混淆矩阵差异

图 5.9 给出两类方法在混淆矩阵均值上的逐格差值。据此可归纳为：

- (1) Kelihos_ver1、Gatak 与 Vundo 的对角线整体抬升较为明显：与表 5.3 中该类 F1 的增益方向相符。
- (2) Ramnit $\rightarrow$ Tracur 方向的误分配有所回落：刻画了结构相近家族之间边界的微调效果。
- (3) Simda 的非对角线与对角线增减在不同区域交错：这与第 5.4 节中对极少数类评测高方差的分析相一致，仅凭单一混淆矩阵差分图不宜过度解读。

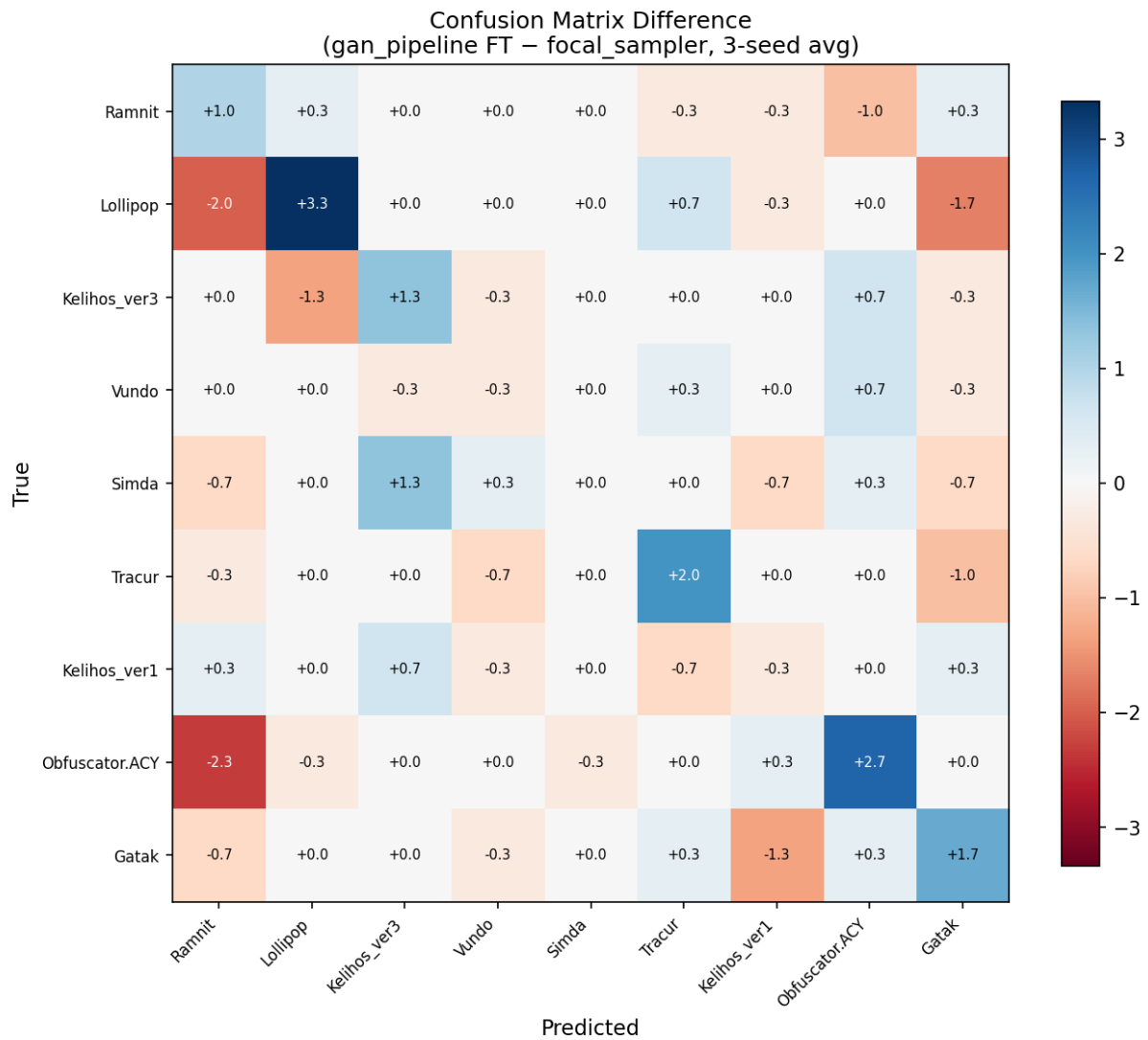


图 5.9 gan_pipeline (Finetune) 与 focal_sampler 混淆矩阵差异热力图 (3 种子均值; 数据源: results/analysis/confusion_diff_ft_vs_focal.png)

5.8 消融实验结果

在核心实验基础上，本文进一步对完整管线进行了消融分析。所有结果基于 3 个随机种子的均值与标准差，汇总于表 5.8：

表 5.8 消融实验结果 ($n=3$, 均值 $\pm$ 标准差)

变体	Accuracy	Macro-F1	Macro-Recall	Weighted-F1
gan_pipeline (GAN)	0.9434 ± 0.0093	0.8575 ± 0.0142	0.8484 ± 0.0082	0.9432 ± 0.0089
gan_pipeline (FT)	0.9446 ± 0.0077	0.8583 ± 0.0168	0.8511 ± 0.0115	0.9446 ± 0.0072
gan_no_fm	0.9430 ± 0.0097	0.8573 ± 0.0140	0.8481 ± 0.0077	0.9429 ± 0.0091
gan_no_gp	0.9434 ± 0.0097	0.8581 ± 0.0137	0.8486 ± 0.0081	0.9432 ± 0.0093
gan_unfrozen_backbone	0.8950 ± 0.0156	0.8077 ± 0.0048	0.8009 ± 0.0043	0.8967 ± 0.0150
finetune_no_filter	0.9448 ± 0.0096	0.8594 ± 0.0156	0.8518 ± 0.0103	0.9449 ± 0.0090
gan_only	0.9434 ± 0.0093	0.8575 ± 0.0142	0.8484 ± 0.0082	0.9432 ± 0.0089

图 5.10 以热力形式对比各消融变体在九个家族上的平均逐类表现，可作为表 5.8 总体指标的逐类佐证。

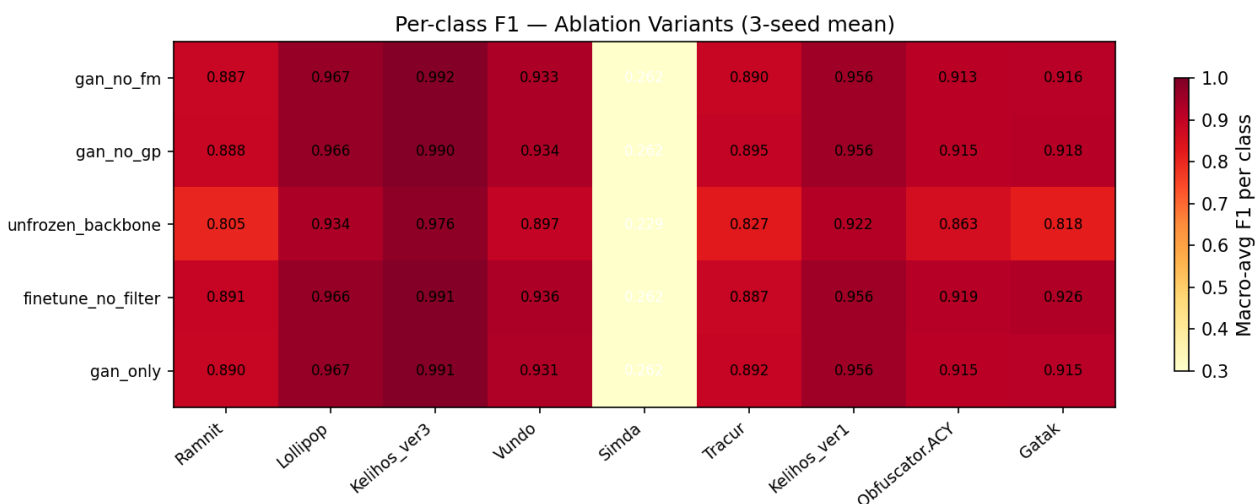


图 5.10 消融变体与各阶段模型在各恶意软件家族上的逐类平均 F1 热力图 ($n=3$ 种子均值；数据源：results/analysis/heatmap_ablations.png)

消融实验给出以下五点关键观察：

- (1) **冻结骨干网络是当前管线中最关键的稳定性因素。**当骨干网络不冻结时(gan_unfrozen_backbone) macro-F1 降至 0.8077，降幅达 -0.0498 ，说明在长字节序列与有限样本设定下，联合更新全部 MalConv 参数会破坏已有判别表示；
- (2) **特征匹配贡献体现在训练初期稳定化。**去除 FM 后 macro-F1 仅变化 -0.0002 ，结合训练曲线分析，FM 的主要价值在于加速生成器特征分布向真实分布的对齐；

- (3) **梯度惩罚的影响有限**。去除 GP 后 macro-F1 变化 +0.0006，其主要作用更可能体现在训练过程的平滑性保障而非最终指标提升；
- (4) **增强微调的边际收益小**。gan_only 与完整 gan_pipeline 的 macro-F1 仅相差约 +0.0008，说明多数收益已在 GAN 阶段获得，后续微调主要起到小幅稳固作用；
- (5) **样本过滤策略的定位需重新审视**。finetune_no_filter 的 macro-F1 为 0.8594，略高于带过滤版本的 0.8583。这提示固定阈值的过滤策略在当前设定下可能过于保守，未来研究可探索自适应阈值或基于分类器不确定性的动态过滤机制。

5.9 讨论：生成增强的收益边界

综合 5.2 至 5.8 节的全部分析，本文将生成增强在长尾恶意软件家族分类中的收益边界总结如下：

- (1) **对 macro-F1 均值的影响有限**。10 种子下 gan_pipeline (FT) 相较 focal_sampler 的 macro-F1 增幅为 +0.0022，Cohen's $d = +0.12$ （微小效应量），未达统计显著性；其均值仍低于更简单的 raw_malconv ($\Delta = -0.0142$, Wilcoxon $p = 0.010$)，后者在 9 类中的 7 类上取得均值最优。
- (2) **中等类家族存在可重复正向增益**。10 种子下 GAN 管线在 Kelihos_ver1、Vundo、Gatak、Tracur 等样本量 400–1500 的中等类上对 focal_sampler 取得方向一致的提升，呈现“甜蜜区间”假说。
- (3) **极少数类 Simda 未受益于生成增强**。在 42 个训练样本的极端少样本设定下，类条件生成器尚不能产出足以提升判别边界的有效合成样本，10 种子均值反而由 0.414 降至 0.367。
- (4) **跨种子稳定性显著提升**。gan_pipeline (FT) 的 macro-F1 标准差较 raw_malconv 下降约 44%，macro-Recall 标准差下降约 59%，呈现“略低均值-显著低方差”特征，在追求部署稳定性的安全场景中更具实用潜力。
- (5) **核心组件层面的关键发现**。冻结骨干网络是管线有效性的核心条件，特征匹配、梯度惩罚与置信度过滤对总体均值的影响有限。

5.10 本章小结

本章按第四章定义的协议系统报告了本文方法的实验结果。首先通过张量级哈希审计确认了旧版 GAN 实现的实验设计错位并给出修复证据；其次在 10 种子下报告了 5 种核心方法

的总体与逐类指标，并通过配对 t 检验、Wilcoxon 符号秩检验与 Cohen's d 效应量分析评估了关键方法对的统计显著性；接着从跨种子方差与混淆矩阵差异角度刻画了生成增强的稳定性优势；最后通过 5 个消融变体分析了各组件的边际贡献。综合而言，本文方法在 macro-F1 均值上未对简单基线 raw_malconv 形成统计显著优势，但其跨种子方差显著更低，且冻结骨干策略被实证为管线最关键的稳定性因素。下一章将基于这些结果对全文工作进行总结，并展望后续可能的研究方向。

第六章 总结与展望

6.1 本文工作总结

针对长尾恶意软件家族分类中“极端类别不均衡”与“生成增强收益不可重复”两个核心问题，本文以字节级 MalConv 为骨干，提出了一种结合条件生成、对抗训练与增强微调的统一框架，并建立了面向多种子、多指标、多统计检验的可复现实验协议。本文的主要工作可概括如下：

- (1) **框架层面**。提出条件 GAN 增强 MalConv 框架，引入类别条件软字节生成器、共享 MalConv 骨干的辅助分类判别器，并通过梯度惩罚与特征匹配机制实现对抗训练的稳定化；设计基于分类置信度的合成样本过滤与按类目标数补齐策略，使生成样本服务于少数类补强而非无差别扩增；明确以“冻结骨干 + 在 GAN 阶段允许更新分类头”的训练协议替代旧版“分类张量被旁路”的实验设计错位。
- (2) **协议层面**。构建包含多随机种子训练（核心实验 $n = 10$ 、消融实验 $n = 3$ ）、逐类指标导出、关键组件消融分析、配对 t 检验、Wilcoxon 符号秩检验、Cohen's d 效应量与张量级 checkpoint 审计在内的可复现实验协议；通过统一入口脚本 `run_paper_experiments.py` 与聚合脚本 `paper/aggregate_10seed_for_paper.py` 实现“训练—评估—汇总—统计”全流程的可复现归档。
- (3) **结论层面**。基于 10 种子扩展实验得到以下三点核心结论：
 - (a) GAN + Finetune 相较直接长尾基线 `focal_sampler` 在 `macro-F1` 上变化 $+0.0022$ （配对 t 检验 $p = 0.731$ ，Cohen's $d = +0.12$ ），未达统计显著性，但跨种子标准差由 0.0261 降至 0.0173 ，呈现一致的稳定性改进；
 - (b) 本文方法的均值性能未超过更简单的 `raw_malconv` ($\Delta = -0.0142$ ，Wilcoxon $p = 0.010$)，后者反而在 9 类中的 7 类上取得 10 种子均值最优；
 - (c) 在极少数类 Simda 上未观察到可重复的生成增强收益，相对而言简单的 `weighted_ce` 给出了最稳健的 Simda 均值。
- (4) **方法学层面**。本文工作通过对历史实验工件的张量级审计，揭示并修复了旧版 GAN 实现中“分类张量未真正更新即被评估”的实验设计错位；通过将随机种子由 3 个扩展至 10 个，对原 3 种子下“GAN 在 Simda 上小幅提升、`focal_sampler` $\rightarrow$ `gan_pipeline` 取得中等效应量”等乐观结论进行了重要修正，凸显了多种子统计在长尾稀少类评估中的必要性。

综合而言,本文提出的框架在“均值-方差”双维度下呈现“略低均值-显著低方差”的特征,更适合被定位为“长尾恶意软件分类中的生成增强机制研究工具”,其价值不在于刷新单一指标的最优值,而在于通过严谨的多维评估系统刻画 GAN 在安全长尾场景中的真实行为边界。

6.2 进一步研究方向

基于本文的实验观察,未来可在以下方向开展进一步研究:

- (1) **面向极少数类的小样本学习。** Simda 等极少数类(约 42 个样本)在测试集中仅 3–6 个样本,其评估指标内在不稳定。后续可探索基于度量学习(如原型网络、孪生网络)、迁移学习或 few-shot 学习范式的方法,在不依赖大规模生成增强的前提下改善极少数类识别。
- (2) **自适应过滤与不确定性建模。** 本文中固定阈值 $\tau_{conf} = 0.5$ 的置信度过滤被实证为略偏保守,后续可探索基于分类器熵、Monte Carlo Dropout 不确定性或样本难度的自适应过滤策略,并对增强样本进行加权而非二元接受/拒绝。
- (3) **更具语义保真的字节生成器。** 本文生成器输出的是经 padding 处理后的 8192 字节短序列,对真实 PE 文件中节区结构、IAT 等语义对象的保真度仍有限。后续可结合 PE 结构先验、扩散模型或 token 级别的离散流模型,提升生成样本的语义一致性。
- (4) **跨数据集泛化与对抗鲁棒性评估。** 本文实验仅基于 Microsoft MMCC 单一公开数据集。后续应在 EMBER^[4]、SOREL-20M^[27] 等更大规模或时间序列分布漂移数据集上评估方法的泛化能力,并考察在对抗扰动(如 MalGAN^[19] 风格逃逸攻击)下的鲁棒性变化。
- (5) **“稳定性-性能”联合优化方法设计。** 本文最重要的实证发现之一是 gan_pipeline (FT) 在均值上不及 raw_malconv、但跨种子方差显著更低。后续可显式将跨种子方差或最差种子性能(worst-seed performance)作为优化目标,例如通过种子内集成、Sharpness-Aware Minimization 或 PAC-Bayes 风格的稳定性正则,构建对部署稳定性更友好的安全分类系统。
- (6) **统计功效进一步强化。** 本文的关键对比已使用 10 种子配对检验,但部分对比仍只能给出边际或微小效应量结论。进一步缩小置信区间需要更大规模的种子(如 30 以上)或基于 bootstrap 重抽样的评估,未来可在算力允许的条件下补充。

总的来看,生成对抗机制在长尾恶意软件家族分类中并非“万能解”,其收益高度依赖类别样本量、骨干稳定性与评估协议的严谨性。希望本文在方法、实验协议与诚实结论三个层

面的工作，能够为后续安全机器学习社区开展更可靠、更可重复的生成增强研究提供一份具体的实证参考。

参考文献

- [1] AV-TEST Institute. Malware Statistics & Trends Report[M]. [S.l.: s.n.], 2023.
- [2] Ucci D, Aniello L, Baldoni R. Survey of Machine Learning Techniques for Malware Analysis[J]. Computers & Security, 2019, 81: 123–147.
- [3] Raff E, Barker J, Sylvester J, et al. Malware Detection by Eating a Whole EXE[C]. Proceedings of the AAAI Workshop on Artificial Intelligence for Cyber Security (AICS). [S.l.: s.n.], 2018: 268–276.
- [4] Anderson H S, Roth P. EMBER: An Open Dataset for Training Static PE Malware Machine Learning Models [M]. [S.l.: s.n.], 2018.
- [5] Ronen R, Radu M, Feuerstein C, et al. Microsoft Malware Classification Challenge[M]. [S.l.: s.n.], 2018.
- [6] Lin T Y, Goyal P, Girshick R, et al. Focal Loss for Dense Object Detection[C]. Proceedings of the IEEE International Conference on Computer Vision (ICCV). 2017: 2980–2988.
- [7] Zhang Y, Kang B, Hooi B, et al. Deep Long-Tailed Learning: A Survey[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2023, 45(9): 10795–10816.
- [8] Mirza M, Osindero S. Conditional Generative Adversarial Nets[M]. [S.l.: s.n.], 2014.
- [9] Goodfellow I J, Pouget-Abadie J, Mirza M, et al. Generative Adversarial Nets[C]. Advances in Neural Information Processing Systems (NeurIPS): volume 27. [S.l.: s.n.], 2014: 2672–2680.
- [10] Odena A, Olah C, Shlens J. Conditional Image Synthesis with Auxiliary Classifier GANs[C]. Proceedings of the 34th International Conference on Machine Learning (ICML). [S.l.: s.n.], 2017: 2642–2651.
- [11] Krčál M, Švec O, Bálek M, et al. Deep Convolutional Malware Classifiers Can Learn from Raw Executables and Labels Only[C]. ICLR 2018 Workshop. [S.l.: s.n.], 2018.
- [12] Coull S E, Gardner C. Activation Analysis of a Byte-Based Deep Neural Network for Malware Classification [J]. IEEE Security and Privacy Workshops (SPW), 2019: 21–27.
- [13] Raff E, Fleshman W, Zak R, et al. Classifying Sequences of Extreme Length with Constant Memory Applied to Malware Detection[J]. Proceedings of the AAAI Conference on Artificial Intelligence, 2021, 35(11): 9386–9394.
- [14] Cui Y, Jia M, Lin T Y, et al. Class-Balanced Loss Based on Effective Number of Samples[C]. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2019: 9268–9277.
- [15] Cao K, Wei C, Gaidon A, et al. Learning Imbalanced Datasets with Label-Distribution-Aware Margin Loss[C]. Advances in Neural Information Processing Systems (NeurIPS): volume 32. [S.l.: s.n.], 2019: 1567–1578.
- [16] Salimans T, Goodfellow I, Zaremba W, et al. Improved Techniques for Training GANs[C]. Advances in Neural Information Processing Systems (NeurIPS): volume 29. [S.l.: s.n.], 2016: 2234–2242.
- [17] Gulrajani I, Ahmed F, Arjovsky M, et al. Improved Training of Wasserstein GANs[C]. Advances in Neural Information Processing Systems (NeurIPS): volume 30. [S.l.: s.n.], 2017: 5767–5777.

- [18] Jang E, Gu S, Poole B. Categorical Reparameterization with Gumbel-Softmax[C]. Proceedings of the 5th International Conference on Learning Representations (ICLR). [S.l.: s.n.], 2017.
- [19] Hu W, Tan Y. Generating Adversarial Malware Examples for Black-Box Attacks Based on GAN[M]. [S.l.: s.n.], 2017.
- [20] Liu J, Zhang Y, Li H, et al. GAPGAN: Adversarial Patch Attacks against Deep Learning Models[M]. [S.l.: s.n.], 2019.
- [21] Anderson H S, Kharkar A, Filar B, et al. Learning to Evade Static PE Machine Learning Malware Models via Reinforcement Learning[M]. [S.l.: s.n.], 2018.
- [22] Pineau J, Vincent-Lamarre P, Sinha K, et al. Improving Reproducibility in Machine Learning Research (A Report from the NeurIPS 2019 Reproducibility Program)[C]. Journal of Machine Learning Research: volume 22. [S.l.: s.n.], 2021: 1–20.
- [23] Henderson P, Islam R, Bachman P, et al. Deep Reinforcement Learning that Matters[C]. Proceedings of the AAAI Conference on Artificial Intelligence: volume 32. [S.l.: s.n.], 2018: 3207–3214.
- [24] Arp D, Quiring E, Pendlebury F, et al. Dos and Don'ts of Machine Learning in Computer Security[J]. USENIX Security Symposium, 2022: 3971–3988.
- [25] Dauphin Y N, Fan A, Auli M, et al. Language Modeling with Gated Convolutional Networks[C]. Proceedings of the 34th International Conference on Machine Learning (ICML). [S.l.: s.n.], 2017: 933–941.
- [26] Wilcoxon F. Individual Comparisons by Ranking Methods[J]. Biometrics Bulletin, 1945, 1(6): 80–83.
- [27] Harang R, Rudd E M. SOREL-20M: A Large Scale Benchmark Dataset for Malicious PE Detection[M]. [S.l.: s.n.], 2020.

附录 A 程序清单

第三章 条件 GAN 增强 MalConv 框架核心代码（项目根目录 GAN-MalConv）

`models/malconv.py`: MalConv 骨干网络与 AC-GAN 共享骨干判别器实现, 对应式 2.1–2.4 与 3.2。

`models/generator.py`: 类条件残差上采样字节生成器, 对应 3.3 节生成器结构与 Gumbel-Softmax 输出。

`config.py`: 超参与路径配置, 对应表 3.1 与表 4.3 的全部超参取值。

`losses.py`: Focal Loss、特征匹配损失与梯度惩罚的通用实现, 对应式 2.7 与 2.6。

`dataset.py`: `.bytes` 文件解析、分层抽样划分与 `WeightedRandomSampler` 构造, 对应 4.1.3 节预处理流程。

第四章 训练与评估脚本（项目根目录 GAN-MalConv）

`train_baseline.py`: Phase 1 MalConv 基线训练入口, 支持 `raw_malconv`、`weighted_ce` 与 `focal_sampler` 三种 profile。

`train_gan.py`: Phase 2 对抗训练入口, 支持 `freeze_backbone`、`train_class_head`、`gp_lambda`、`fm_lambda` 等开关, 覆盖核心实验与全部消融变体。

`train_finetune.py`: Phase 3 增强微调入口, 包含基于式 3.8 的置信度过滤与 $N_{aug} = 800$ 按类目标数补齐逻辑。

`evaluate.py`: 测试集统一评估入口, 输出 `metrics.json`、`metrics_summary.csv`、`per_class_metrics.csv` 与混淆矩阵图。

`audit_checkpoints.py`: checkpoint 张量级哈希审计工具, 对应 5.1.1 节表 5.1 的审计结果。

`run_paper_experiments.py`: 多种子、多 profile 实验编排入口, 统一调度 `train_baseline`、`train_gan`、`train_finetune` 与 `evaluate`。

`experiment_utils.py`: 跨脚本共用的种子设置、目录管理与日志工具。

第五章 结果聚合与可视化脚本（`paper/` 与 `results/analysis/`）

`paper/aggregate_10seed_for_paper.py`: 10 种子核心结果聚合脚本, 输出 `aggregate_10seed_for_paper.json` 与 `aggregate_10seed_for_paper.md`, 对应表 5.2、表 5.3 与表 5.4。

`analyze_experiments.py`: 跨方法可视化与统计检验脚本, 输出 `heatmap_core.png`、`heatmap_ablations.png`、`training_dynamics.png`、`finetune_dynamics.png`、`seed_variance_boxplot.png`、`simda_fl_comparison.png`、

`confusion_diff_ft_vs_focal.png`、`statistical_tests.json` 与 `simda_analysis.json`, 对应 5.5、5.6、5.7 节的全部分析图与统计表。

附录 B 攻读专业学位硕士期间撰写的论文

- [1] 沈文强. 面向长尾恶意软件家族分类的条件 GAN 增强 MalConv 框架研究 [D]. 南京: 南京邮电大学, 2026.
- [2] (拟投) **Shen W**, et al. Toward Reproducible Generative Augmentation in Long-Tailed Malware Family Classification: A Multi-Seed Empirical Study on MalConv[J]. *IEEE Transactions on Information Forensics and Security*, 2026. (撰稿中)

附录 C 攻读专业学位硕士期间申请的专利

[1] (暂无) 攻读硕士学位期间未申请发明专利。

附录 D 攻读专业学位硕士期间参加的科研项目

- [1] (示例, 请按实际参与项目修改) 江苏省研究生科研创新计划, 面向长尾恶意软件分类的生成增强机制研究 (XXXXXX)。

致谢

时光荏苒，转眼间硕士阶段的学习与研究即将告一段落。论文落笔的此刻，谨向所有在求学路上给予我帮助与陪伴的师长、朋友与家人致以最诚挚的谢意。

首先衷心感谢我的导师 XXX 教授。从研究方向的选择、研究问题的凝练，到实验设计、论文写作与答辩准备的全过程，导师始终以严谨求实的治学态度与精益求精的学术追求悉心指导，使我在面对“GAN 增强不显著”这一负向结果时仍能保持研究定力，最终将其凝练为一份诚实而有方法学价值的论文工作。导师在多种子统计协议、消融实验严谨性与论文写作约束上的反复叮嘱，让我深刻体会到一个研究者对“可复现性”应有的敬畏与责任。

感谢实验室各位师兄师姐、同门同学在算力调度、数据获取与代码 debug 过程中的无私帮助。在 GAN 训练长期不收敛、checkpoint 张量审计揭示历史 bug 的关键节点，他们的耐心讨论与代码 review 让我度过了许多个艰难的夜晚。

感谢南京邮电大学通信与信息工程学院（或所属学院）提供的良好科研环境与计算资源，感谢学院与研究生院在论文撰写与答辩组织过程中给予的细致服务。

感谢父母多年来对我求学之路的默默支持与无条件信任。无论我做出怎样的研究选择，他们都给予我充足的时间与空间去尝试与试错，这是我能够安心完成本论文最坚实的后盾。

最后，感谢所有评阅本论文的专家学者，您的宝贵意见将是我在网络空间安全研究道路上继续前行的重要指引。